

Types of apps

- 1] Mobile applications
- 2] Web applications
- 3] Stand alone applications
- 4] Business applications

1] Mobile applications - Spotify, twitter, netflix, prime, Byju's Learning app.

2] Web applications - Book my show, canva, Gmail, make my trip, pininterest.

3] Stand alone applications - Calculator, clock, Gallery, Camera, Calender.

4] Business - Tira software, help desk applications, accounts software, Billing software, Database software.

JVM - It is element of JDK software which is used to
(Java virtual machine) identify name of elements like class
Variable, packages, methods and interfaces
etc

JRE will call only the main() for execution.

1. Basics of JAVA

classmate

Date _____
Page _____

programming language - (PL)

programming lan. it is a set of instructions to perform a particular task by machine which is given by the programmer

programming lan. is classified into 3 types

- 1] High level PL
- 2] Low level PL
- 3] Middle level PL

1] High level programming language -

The lang. which is writable, readable, understandable and executable by the programmers is called as High Level programming lan.

e.g - python, c#, java, ruby etc.

2] Low Level programming language -

1] The Language which is understandable and executable by the machines is called as Low Level prog. La.

ex - Binary language (0, 1)

2] Low Level PL it is also called as Machine Level PL.

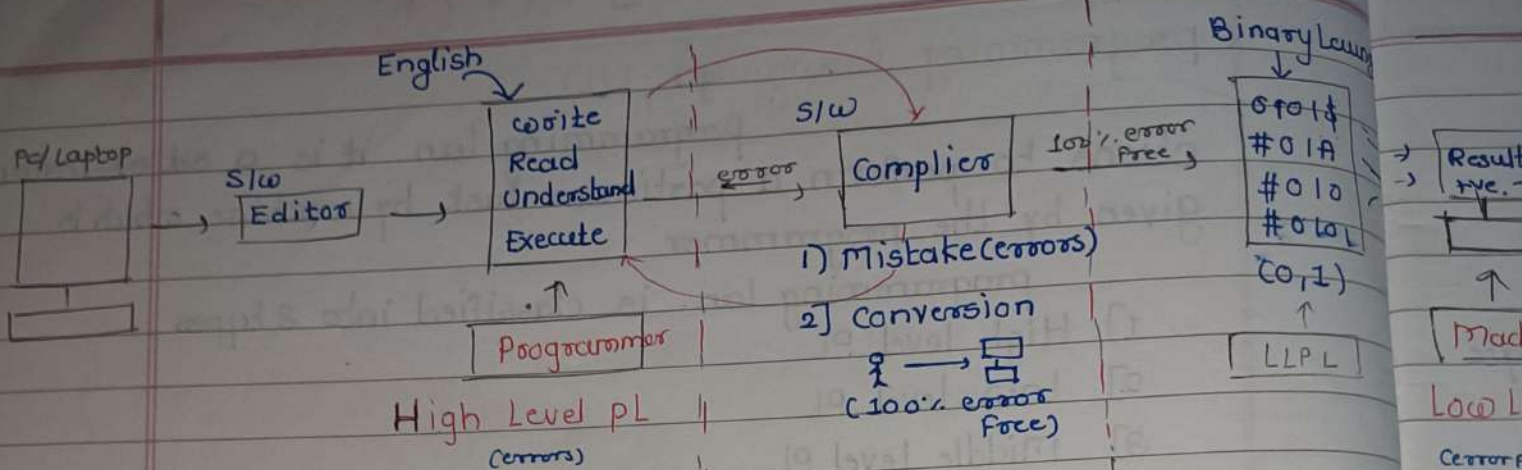
3] Middle Level programming language -

It is a computer programming language that interacts with abstraction layer of computer system.

ex - C++, java byte code.

OR

the language which converts source code into machine code for execution by abstraction layer.



Que. What is Java or what are the features of Java or what are the characteristics of Java

- 1] Java is high level programming language.
- 2] Java is easy to understand (for technical features).
- 3] Java provides high security.
- 4] Java is platform independent.
- 5] Java is open source (free to use).
- 6] Java is object oriented programming language.
- 7] Java supports packages.
- 8] Java supports inbuilt libraries.

Que - Why java is called as platform independent?

- 1] A source code which is created using Java:
- It is not directly converted into Machine level programming language (Binary) instead of it is converted into intermediate language called Byte code.
- 2] We can execute these byte code in any platform which has JDK (Java development kit) software installed.
- 3] Hence java is called as platform independent but it is totally dependent on JDK software.

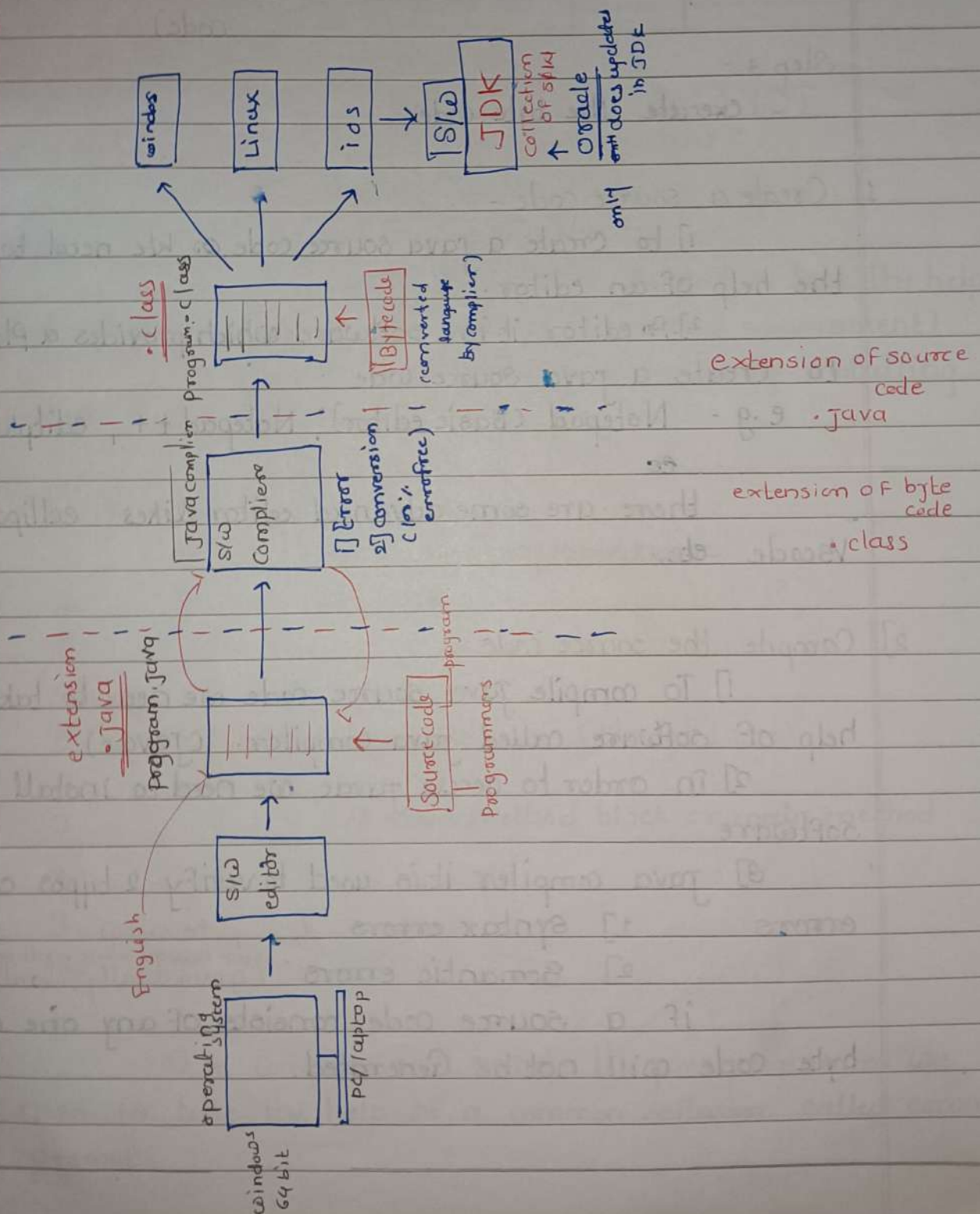
Note - We cannot execute ~~any~~ java programme in any machine or any platform which ^{has} no JDK software (install it's package of small elements)

Result
+ve, -ve

Machin^g

Low Level
PL
(Error Free)

• extensions are mandatory to see the programme



* Steps to execute a java programme.

→ Step 1 -

Create a source code

Step 2 -

Compile the source code (source code is converted to byte code)

Step 3 -

execute the byte code

1] Create a source code -

1] to create a java source code we need to take the help of an editor.

2] An editor it is software which provides a platform to create a java source code

e.g - Notepad (Basic editor), Notepad ++, editplus (advanced)

there are some advanced editor likes eclipse, Vscode etc.

2] Compile the source code -

1] To compile java source code we need to take the help of software called java compiler. (javac)

2] In order to access javac we need to install JDK software

3] java compiler it is used to verify 2 types of errors

1] Syntax errors (, , missing)

2] Semantic errors - error happens while writing logic (grammatically, word error)

if a source code consists of any one error byte code will not be Generated.

2] The command to compile the java source code is

Javac class_name.java

stands for java compiler

program name

extension

(must save as .java)

3] the command to execute the java byte code or class file

java class_name - class name

program name

* TOKENS -

tokens are the smallest unit of Java programming. tokens of java are classified into

1] keywords

2] identifiers

3] literals

4] separators

1] keywords - keywords are predefined and reserved words of java

We can identify keywords of java in two ways

1] they are highlighted in blue colour.

2] All the keywords of java are written in lowercase.

examples - class, public, static, void etc

Que - Write all the keywords of java -

1] abstract

2] assert

3] boolean

4] Break

5] byte

6] case

7] Catch

8] char

9] class

10] Continue

11] default

12] do

13] double

14] else

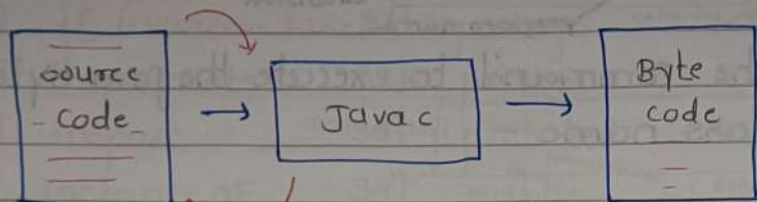
15] enum

16] extends

17] final

18] finally

3] When the source code is 100% errorfree (0 syntax and semantic error) then Source code is converted into bytecode.



1] errors -

1] syntax errors

2] semantic errors

2] Converted (100% errorfree)

3] Execute the byte code -

1] To execute java byte code we need to take the help of software called JRE (java Runtime environment)

2] We can access the JRE software by installing JDK

* Structure of Java

1. Class class_name/ programme_name

2. { // class block

3

4

5 public static void main (String [] args)

6 {

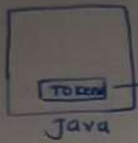
7 // main method block or main method

8 }

9 }

10

1] To compile and execute the java programme we need to take the help of a common software called command prompt



smallest unit

Java

classmate

Date

Page

19] float	29] new	39] Super
20] for	30] null	40] switch
21] if	31] package	41] synchronized
22] implements	32] private	42] this
• 23] import	33] protected	43] throw <i>unused keyword</i>
• 24] instanceof	34] public	44] throws
25] int	35] return	• 45] transient
26] interface	36] short	• 46] try
27] long	37] static	47] void
• 28] native	• 38] Strictfp	• 48] volatile
		51] False
		50] True
		52] const
		49] while

2] Identifiers - Identifiers its name Given by the programmer
to elements of java

the elements of java are

- 1] class_name
- 2] Variable_name (variable - name of block of memory. It changes)
- 3] Method_name
- 4] interface ^{face} name
- 5] package name

Ques. What are class naming Conventions

Rule 1 - The first letter of class name can be written in terms of upper case or lower case but these rule it is not mandatory, it totally depends on IT company standards

e.g. Apple.java }
apple.java } *not mandatory*
but most of the time we use uppercase

Rule 2 -

1] We cannot start java, class name with numerical values (we get compile time error).

1Creature.java → CTE ✗

2] In order to use Numerical value in program name or class name we need to follow alphanumeric rules

Creature1.java ✓

C1.java ✓

Rule No. 3 -

1] We cannot give a space between a class name (we get compile time error).

Mango one.java → CTE

2] In order to fill the space we can use two special characters (_ , \$)

Mango_one.java

Mango\$one.java

Rule No. 4 - 1] We cannot use keyword as a class name or program name. (We get compile time error)

Void.java ← CTE

3] Literals

Literals are the different types of values we use in java programming.

It is classified into

1] Values / Numbers

2] Character

3] String

1] Values / Numbers -

1] Integer - It is a whole number

10, 20, 30

2] Float - number which having decimal point

10.11, 12.31, 11.11

2] Character - Character are single element

classified in uppercase, lowercase, special characters

'A' 'b' '\$'

characters are written in single code (always) identification for machine

ASCII values
← A a
↓ ↓
65 97

3] (words) String - "Apple" "klippro"

Strings are written in double code always.

Collection of characters

4] boolean - True

False

4] Separators - , , ; , { }

{ - used to define starting of block.

} - used to define ending of block

()	{ }	[]	;	,	.	:
Parenthesis	Braces	Bracket	semicolon	comma	period	Colons

* printing statements of java -

printing statements of java is classified into two types.

1] `System.out.println()`

2] `System.out.print()`

1] `System.out.println()` -

- It is used to print value and move the cursor to next new line

- For `println` statement passing a value is not mandatory. Only value not task variable name, mcs)
e.g - 100, 20, 30

Class P1

{

public static void main (String [] args)

{

`System.out.println(1);`

`System.out.println(2);`

}

}

main

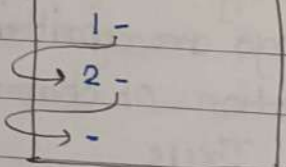
{

`println(1);`

`println(2);`

}

CHD



O/p = 1

2

String can merge anybody anybody

ASCII value concept only for character not for string

Date _____
Page _____

3rd words depend on 1st and 2nd word

2] System.out.print() :-

- It is used to print value & move the cursor on the same line.
- print statement works like word by word.
- For print statement passing a value is mandatory

```

class P1
{
    public static void main (String[] args)
    {
        System.out.print (1);
        System.out.print (2);
    }
}

```

{		
print (1);	CMD	O/p = 1 2 *
print (2);	1 2 -	
}		

* Printing different types of values using printing statements

```
Class P1
{
    Public Static Void main (String [] args)
    {
        // literals
        System.out.println (12);
        System.out.println (12.123);
        System.out.println ('a');
        System.out.println ("abcd");
        System.out.println (true);
        System.out.println (false);
    }
}
```

* ^{help us} Separators - defining the structure of program.

- Search in more depth
- 1] () parentheses - used to contain a list of parameters in method definition, surrounding cast types precedence in Expressions in control statements.
 - 2] { } Braces - used to define a block of code, for classes methods
 - 3] [] Bracket - declare array types
used when dereferencing array values.
 - 4] ; semicolon - Terminates statements.
 - 5] , Comma - used to separate package names from sub-package and class names and also selects a field or method from an object
 - 6] . Period - reference.
 - 7] : Colon - used after labels

or
ASCII
value
required
and character
have ASCII
values

For addition no are required (Integer)
For concat string are required

classmate
Date _____
Page _____
07/12/23

- * **+ plus operator** - plus operator it works in two ways
- 1] addition - no are required or ASCII values
 - 2] Concat (string) - string required.

Class P1

```
{
    public static void main(String[] args)
    {
        System.out.println (1+1);
        " " (1+'a');
        " " ('d'+'a');
        " " ("1+1");
        " " (1+"a");
        " " ('a'+"a");
        " " ("1"+true);
        " " (true+true);
        " " ("1"+"a"+true+"abcd"+"a"+123);
        " " ('a'+1+"123"+true+12+false);
        " " ('a'+1+true+"abcd"+1);
    }
```

output

int + int = int
int + character = int
 ↓
 ASCII value
cha + cha = int
 ↓ ↓
 ASCII ASCII
→ String
int/No + string = string
★ String can merge anything,
any value, any special
character everything
string + boolean = string
CTE error B+B = error

first these 2
go for task
Result + 2
Result

20 + '2' character = 70 output

then it will take ASCII
examples

Results (output)

- * Float + integer = Float
Float + character = Float
Float + Float = Float
Character + Float = Float
- 1.5 + 1 = 2.5
1.5 + 'a' = 98.5
 ↑
 ASCII value = 97
1.5 + 1.5 = 3.0

- Strings and Boolean does not have ASCII value
only characters have ASCII values

String + int = String
 String + char = String
 String + Float = String
 String + String = String
 String + Boolean = String
 Boolean + characters = error
 Boolean + integers = error
 Boolean + Float = error
 Boolean + Boolean = error
 Boolean + String = String

"ab" + 1 = ab1
 "ab" + 'c' = abc
 "ab" + 1.5 = ab1.5
 "ab" + "ab" = abab
 "ab" + true = abtrue

* Steps to create directory.

- 1] edit plus open → new → java → structure open.
- 2] open cmd → cd .. \.. ↓
 mkdir wccm16java ↓
 cd wtdp
 cls (minimize)
- 3] open edit plus → C: window ↓
 wcc16java
 Ctrl+S to structure of java
- 4] open cmd ↓
 java c abcd.java
 Java abcd

* Datatypes

1] Datatypes are the type of value we use in java programming

2] Datatypes depends on Literals.

3] Based on Literals datatypes is classified into 2 types.

1] primitive datatype

2] Non-primitive datatype.

1] Primitive datatype -

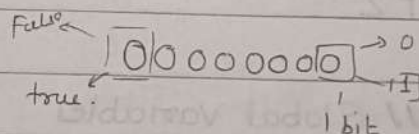
primitive datatype are the type of primitive values we use in java programming.

• primitive datatypes are classified into

keywords

No.	Types of values	Primitive Data type	Size
1]	integer	byte short int Long	1 byte 2 byte 4 byte 8 byte
2]	Float	Float double	4 byte 8 byte
3]	Character	char	2 byte
4]	Boolean	Boolean	1 bit

1 byte = 8 bit



130

Byte + Byte = long int

Short + Short = int

Non-primitive datatypes are not keywords
Comes in Red color

Date _____
Page _____

2] Non-primitive datatype

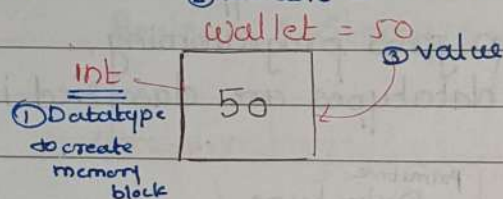
Non-primitive datatype are the type of non-primitive values we use in java programming.

e.g - String. (there some other also)

e.g - class, Array, Interface, object, String. *(written in red color)*

* Variable name -

a variable it's name given to a block of memory to store a value



Variables -

Variable are classified into two types

1] Global variable

2] Local variable

1] Global variable -

Global variables are declared inside the class blocks

(It will access from anywhere, goes in main method block also)

2] Local variable

Local variable are declared inside the method block.

```
class P2  
{
```

```
    // Global variables
```

```
    public static void main (String[] args)
```

```
    {
```

```
        // Local variables
```

```
    }
```

```
}
```

Variable - Datatype, V.N

Semicolon - termination / stopping, end
comma - continue, next

Date _____
Page _____

Syntax of Variable -

datatype variable_name = value;

e.g -

int number = 123;

- 1] Local variables are declared on inside the method block
- 2] We cannot access a local variable outside the method
- 3] Local variable does not consist of default values
- 4] In order to access local variable, Initialization is mandatory (passing value)
- 5] Before initialization we need to declare a datatype and variable name first
- 6] We can declare more than one local variable inside the method block but variable name has to be different.
- 7] Inside method blocks, we cannot declare more than one local variables with same variable name

①

```
class
{
    public static void main (String [] args)
    {
        int wallet = 200;
        System.out.println (wallet); // 200
    }
}
```

②

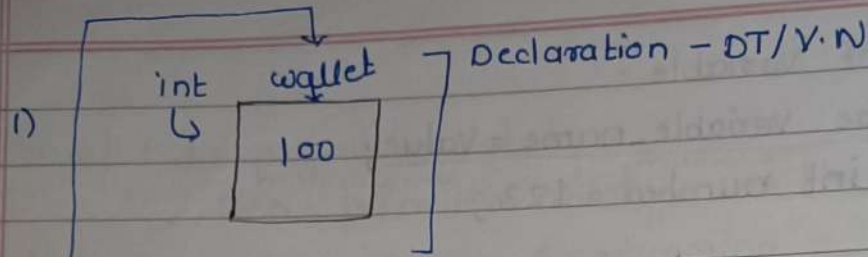
```
{
    int wallet;
    wallet = 200;
    S.O.P (wallet);
}
```

declaration
(datatype, variable Name)

Initialization
(put a value)

default

// 200



2) `wallet=100;` → Initialization - value

3) `SOP(wallet);` 100

```

(3) {   int wallet = 100;
        int wallet = 200;
        // variable name (wallet) has to be different
        S.O.Pln(wallet); // CTE compile time error
    }
    
```

```

(4) {   int wallet = 100;
        int wallet1 = wallet;
        // passing value from one variable to another variable
        S.O.Pln(wallet); // 100
        S.O.Pln(wallet1); // 100 2times
    }
    
```

Que Can we create local variable for both primitive and non-primitive datatype? Explain with a programme and write it in notebook

→ Yes we can create local variable for primitive as well as non-primitive datatype.

```

Class P1
{
    public static void main (String[] args)
    {
        int wallet = 2;
        char d = 'F';
        boolean g = true;
    }
}
    
```

^{when we have to use}
 Float h = 89;
 String c; ~~String~~
 c = "ghj";
 S.o.P ln (c);
 S.o.P ln (cwallet);
 S.o.P ln ('a' + "sdg");
 S.o.P ln (h);
 S.o.P ln (c);
 S.o.P ln (c);
 S.o.P ln (true + "dfg" + 13453 + 34526);
 }
 }

output
 // ghj
 // 2
 // asdg
 // 89.0 ^{answer in Float}
 // f
 // true
 // dfg1345334526

When we have use Float No.

1]

Class P2

{

public static void main (String [] args)

{

Float a = 100.123F;

System.out.println(a);

a = 123.100F;

System.out.println(a);

}

//123.100

}

2]

Comma

Class P3

{

public static void main (String [] args)

{ int a, b, c;

a = 1;

b = 2;

c = 3;

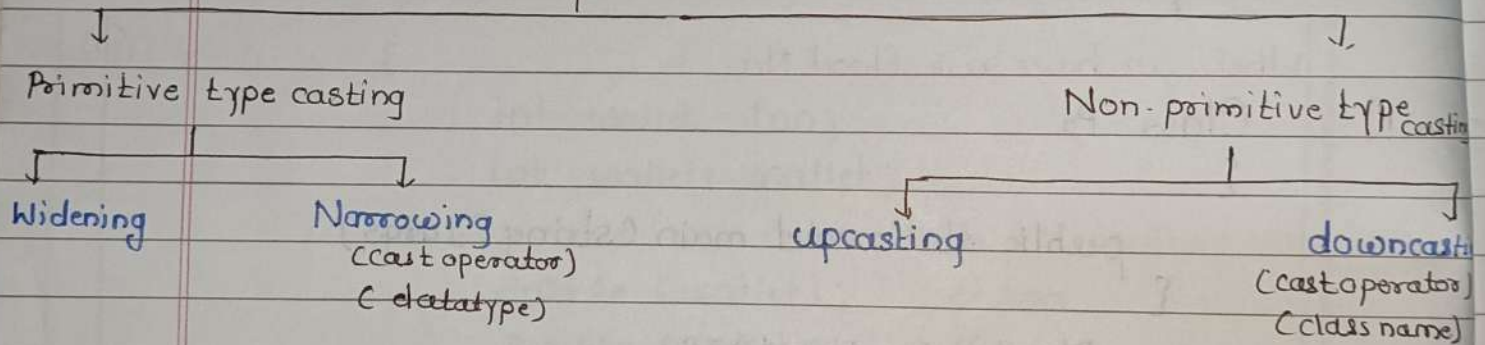
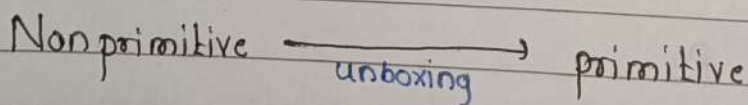
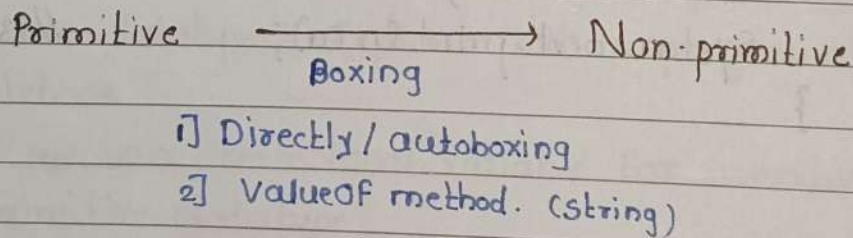
Primitive data type

- 1] int
- 2] char
- 3] byte
- 4] short
- 5] long
- 6] float
- 7] double
- 8] Boolean

Non primitive datatype

- We can create object
- obj-ec-var

- 1] String
- 2] Wrapper classes
- 4]

Type castingAND

Operators

It is a character that represents a specific mathematical or logical action or process.

S.O.P ln (a);

S.O.P ln (b);

S.O.P ln (c);

}

}

* **Operators** - Operators are classified into three types

1] Unary operator

2] Binary operator

3] Ternary operator

1] Unary operator - the operators which takes only 1 value at a time is called as Unary operator

• Unary operator is classified into

1] Increment operator

2] decrement operator

3] CAST operator.

2] Binary operator - the operators which takes (minimum) ^{two} 2 values at a time is called as Binary operator

• Binary operator is classified into

1] Relational operator

2] Arithmetic operator

3] Logical operator

3] Ternary operator - the operators which takes three / 3 ^(min) values at a time is called as ternary operator

• Ternary operator class

1] Conditional operator

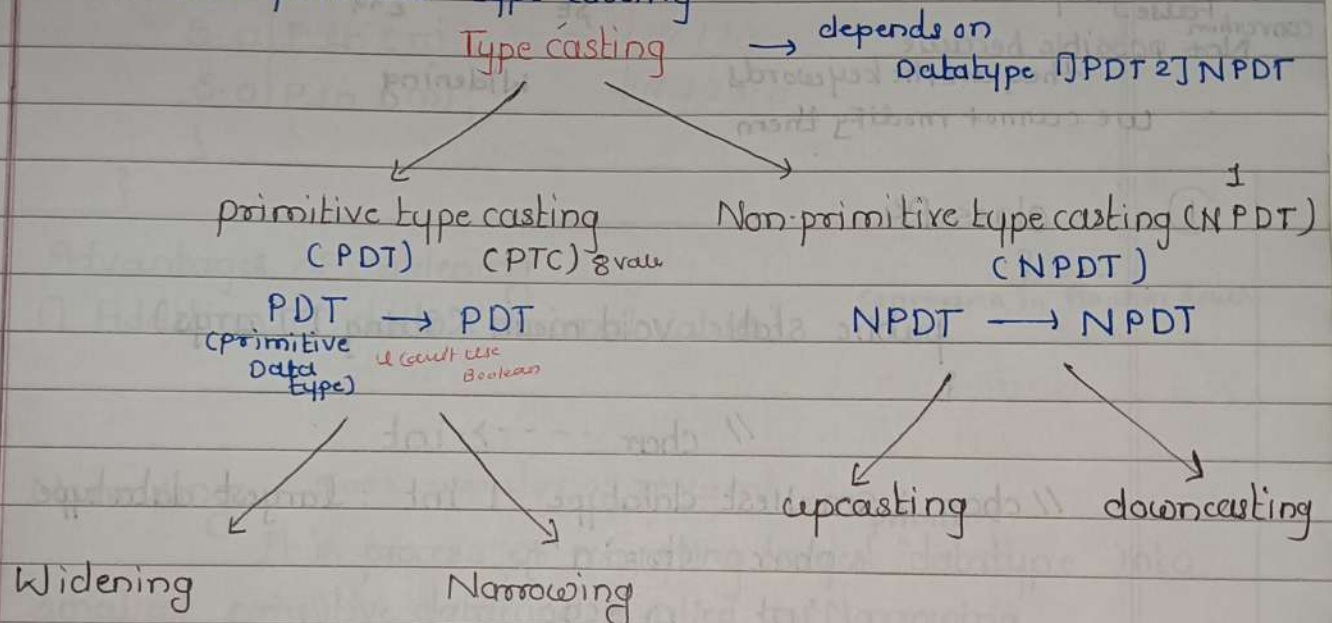
Type Casting

It is a process of converting one datatype into another datatype called as type casting

type casting depends on datatype

type casting is classified into two types

- 1] primitive type casting
- 2] Non-primitive type casting



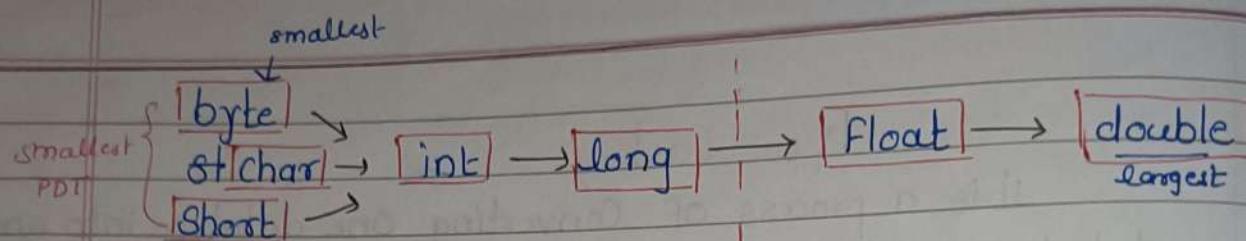
1] primitive type casting -

It's process of converting one primitive datatype into another primitive data type its called as primitive type casting.

It is classified into 2 types

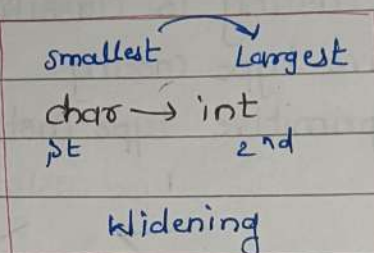
- 1] Widening
- 2] Narrowing

1] Widening - It is a process of converting smallest primitive datatype into largest primitive datatype is called as widening.



Numbers → Integer → Float (dec)

Boolean
True
False } keywords
conversion
Not possible because
they are the keywords
we cannot modify them



①

class P4

{

```

    public static void main (String [] args)
    {

```

```

        // char ----> int

```

```

        // char : smallest datatype | int : largest datatype

```

```

        char ch = 'a';

```

```

        int n = ch;

```

```

        System.out.println (ch);

```

// 'a'

```

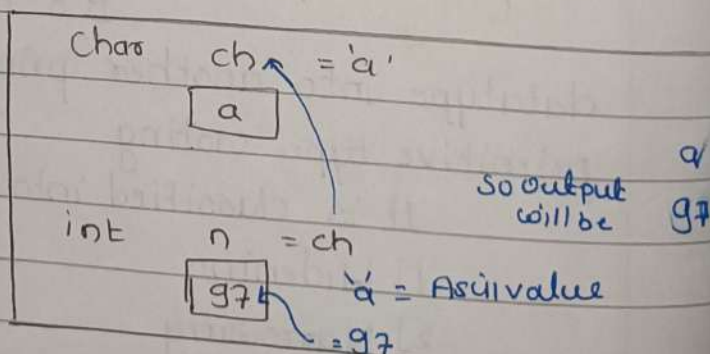
        S.o.Pln (n);

```

// 97

}

}



- Conversion of Boolean (PDT) not possible bec it consist true and false (which are keywords of Java we cannot modify them).

```

② class P5
{
    public static void main(String[] args)
    {
        //int - - - -> double
        int n = 123;
        double m = n;
        S.o.P.println(n); //123
        S.o.P.println(m); //123.0
    }
}

```

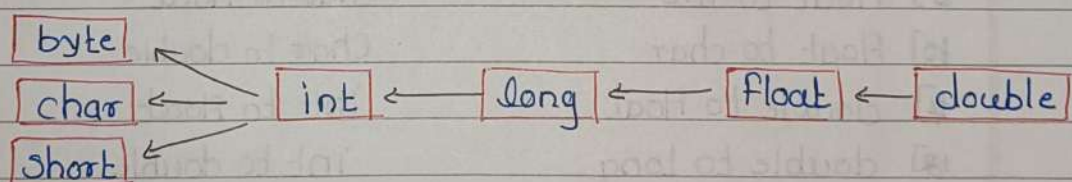
* Advantages of widening.

1] Addition of data (decimal values)

Conversion in float or double

2] Narrowing - Cast operator is required.

It is process of converting largest ^{primitive} datatype into smallest primitive datatype is called as Narrowing.

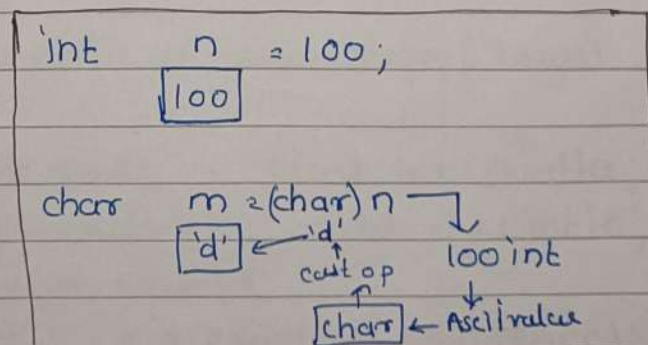


Largest to smallest (conversion)

* disadvantage - 1] loss of data (decimal → whole no.)

① class P6

cast operator
(datatype)



class P6

{

public static void main (String[] args)

{

//int ----> char

int n = 100; // Largest data type

char m = (char)n; // smallest data type

cast operator -> datatype for value

System.out.println(n); //100

System.out.println(m); //d

}

}

Q-1

Narrowing

Widening

Convert 1] int to char

2] Long to int

3] Long to char

4] Float to Long

5] Float to int

6] Float to char

7] double to Float

8] double to Long

9] double to int

10] double to char

Char to Long

Char to int

Char to Long

Char to Float

Char to double

int to Float

int to double

Long to Float

Assignment Questions

classmate

Date _____
Page _____

17 Narrowing (cast operator is required)

Class Narrowing
{

public static void main (String[] args)
{

int a = 23; → char b = (char)a;

Long c = 89; ^{conversion} → int d = (int)c;

↓
char e = (char)c;

Float f = 29; → Long g = (Long)f;

↓
int h = (int)f; → char i = (char)f;

S.o.Pln(a); S.o.Pln(b);

S.o.Pln(c); S.o.Pln(d);

S.o.Pln(e); S.o.Pln(f);

S.o.Pln(g); S.o.Pln(h);

S.o.Pln(i);

}

}

Class Narrowing

{

public static void main (String[] args)

{

double a = 89; Float b = (Float)a;

Long c = (Long)a; int d = (int)a;

char e = (char)a;

S.o.Pln(a); S.o.Pln(b); S.o.Pln(c);

```

        S.o.pln(c);    S.o.pln(e);
    }
}

```

2] widening

```

class widening
{
    &#242; public static void main(String[] args)
    {
        Char -> a = 's';
        Long   b = a;
        Float   c = a;
        double  d = a;

        int     e = 29;
        Float    f = e;
        double   g = e;

        Long Float h = b;

        S.o.pln(a); S.o.pln(b); S.o.pln(c);
        S.o.pln(d); S.o.pln(e); S.o.pln(f);
        S.o.pln(g); S.o.pln(h);
    }
}

```


Value → V - N
Use { storing
printing

6
6
6

classmate

Date _____
Page _____

1] AZ Unary operator -

1] Increment operator -

Increment operator it is used to increment the value which is present inside the variable by 1 and updated

the increment operator is denoted by '++'

increment operator classified in two types

- 1] post increment operator
- 2] pre increment operator

1] post increment operator - the increment operator which is suffixed to its variable name is called as post increment operator.

Syntax -

Variable_name ++

- 2 elements
- 1] variable name (use first)
- 2] increment operator

Steps to perform post increment operator

- 1] Use the value which is present inside the variable first
(storing and printing purposes of value)
- 2] ^{increment} the value which is present inside the variable by one and ^{update}
- 3] Later we will be using updated value.

Class P1

{

public static void main (String[] args) .

{

int a = 1;

S.o.Pln(a); // 1

S.o.Pln(a++); // 1

S.o.Pln(a); // 2

}

}

$a = 1$
12

$a++$
1 ↑
Incr by 1

```

class P2
{
    public static void main (String[] args)
    {
        int a = 1;
        int b = a++;      //2
        S.o.Pln (a);      //2
        S.o.Pln (b);      //1
        S.o.Pln (a);      //2
        S.o.Pln (b++);    //1
    }
}

```

Ques Verify all datatype with post increment operator.

2] pre increment operator -

the increment operator which is prefixed to its variable name is called as pre increment operator
Syntax -

$++$ Variable.name

e.g. $++a;$

Steps to perform pre increment operator -

- 1] Increment the value which is present inside the variable first^{by 1} and update.
- 2] Use the updated value which is present inside the variable.

①

```

class P4
{
    public static void main (String[] args)
    {
        int a=1;    //2
        S.o.P.ln(a);    //1
        S.o.P.ln(++a);    //2
        S.o.P.ln(++a);    //2
    }
}

```

②

```

class P5
{
    public static void main (String[] args)
    {
        int a=5;
        int b=++a;
        int c=++a;
        S.o.P.ln(a);
        S.o.P.ln(b);
        S.o.P.ln(++c);
    }
}

```

Que 3 write the difference between pre and post increment

	Pre increment	Post increment
1]	Increment operator which is prefixed to its variable name	1] Increment operator is suffixed to its variable name.
2]	First var , increment the value inside the variable and update	2] First, use the initial value then increment value present inside variable and update
3]	syntax Variable_name ++ eg- a++;	++ variable_name eg ++a;

In pre increment operator we are able to use updated value first.

In post increment operator we are not able to use updated value on same line.

```

class P5
{
    public static void main (String[] args)
    {
        int a = 2;
        int b = a++;
        int c = b++;

        int res = a++ + b++ + c++ + a++ + b++ + 1;
                                     2       3       2       4       4
                                     ↑
                                     post operator

        System.out.println (res); // 17
    }
}

```

Assignment

Que - Create a notes for decrement operator

2] **Cast operator** - 1] cast operator is used for Narrowing.
2] It is used for converting large primitive datatype into small primitive datatype.

Syntax -

(datatype)

datatype v-name = (datatype)

3] disadvantage is loss of data (Float → int)

2] Decrement operator - Decrement operator it is used to decrement the value which is present inside the variable by 1 and updated.

the decrement operator is denoted by '--'
decrement operator classified in two types

- 1] post decrement operator
- 2] pre decrement operator

1] post decrement operator - the decrement operator which is suffixed to its variable name is called as post decrement operator.

Syntax -

Variable name --	2 elements
	1] variable name (we first)
	2] decrement operator

Steps to perform post decrement operator.

- 1] Use the value which is present inside the variable first (storing and printing purpose of value)
- 2] Decrement the value which is present inside the variable by one and update.
- 3] Later we will be using updated value.

```

① Class P1
{
    public static void main (String [] Args)
    {
        char a = 'S' ;
        char b = a*-- ;

        S.o.Pl n (a);           // S r
        S.o.Pl n (b);           // S
    }
}

```

```

② class P2
{
    public static void main(String[] Args)
    {
        int a = 1;
        int b = a--;
        S.o.P.en (a); // 0
        S.o.P.en (b); // 1
        S.o.P.en (a); // 0
        S.o.P.en (b--); // 1
    }
}

```

2] pre decrement operator -

the decrement operator which is prefixed to its variable name is called as pre decrement operator.

Syntax -

-- variable_name

e.g -- a;

Steps to perform predecrement operator-

1] decrement the value which is present inside the variable First (Storing and printing) by 1 and updated.

2] Use the updated value which is present inside the variable

$b = 2$
 $c = 3$
 $a = 1$
 $a++ + ++c + 4 + 3 + 2 = 15$

```

① class P4
{
    public static void main (String [] args)
    {
        int a = 1;
        S.o.P ln (a); // 1
        S.o.P ln (--a); // 0
        S.o.P ln (a); // 0
    }
}

```

```

② class P5
{
    public static void main (String [] args)
    {
        int a = 5 // 8 4 3
        int b = --a; // 4
        int c = --a; // 3
        S.o.P ln (a); // 3
        S.o.P ln (b); // 4
        S.o.P ln (++c); // 2
    }
}

```

(24)

$4 = a = 4$
 $3 = b = 4$
 $5 = c = 4$

$a++ + --b + a-- + --c + ++c + ++a$
 $b++ + --a + b-- + --b + ++a + ++a$
 $4 + 2 + 5 + 3 + 3 + 4$

(21)

2] Binary operator -

the operator which takes (minimum) 2 values at a time is called as Binary operator

Binary operator is classified into following.

1] Relational operator - Relational operator it is an Binary operator.

1] it take 2 values at a time.

2] the written type (output) of relational operator is a boolean condition (true/false)

```
Class P7
{
    public static void main (String[] args)
    {
        int a = 4;
        int b = 3;
        S.o.p.ln (a > b);           // true
        S.o.p.ln (a < b);           // false
        S.o.p.ln (a >= b);          // true
        S.o.p.ln (a <= b);          // false
        S.o.p.ln (a == b);          // false
        S.o.p.ln (a != b);          // true
    }
}
```

2] Arithmetic operator - It is and Binary operator. it takes 2 values at a time (+, -, *, /, %).

Arithmetic operator it is classified in following types

- 1] Addition 2] subtraction 3] multiplication 4] division 5] modulation.


```

class P8
{
    public static void main (String [] args)
    {
        int a = 4;
        int b = 2;
        S.o.P.ln (a+b);
        S.o.P.ln (a-b);
        S.o.P.ln (a*b);
        S.o.P.ln (a/b);           // Forget Quotient
        S.o.P.ln (a%b);          // For get remainder
    }
}

```

Que - Solve it.

- 1] Write a programme to calculate square of Given number.
- 2] Write a programme to calculate cube of Given number.
- 3] Write a programme to calculate area of rectangle. $W \times L$
- 4] Write a programme to calculate area of circle. πr^2
- 5] Write a programme to calculate area of square. a^2
- 6] WAPT Find the biggest of two Numbers. $a > b$
- 7] WAPT perform addition of three Numbers
- 8] WAPT Find product (*) of ^{three} ~~pre~~ numbers
- 9] WADifference between pre and post decrement operator.
- 10] What is the use of cast operator in java.

Que WAPT Find product (*) of three numbers.

Assignment

Que WAP to perform addition of three numbers

```
class P2
{
    public static void main (String[] args);
    {
        int a=1, b=2, c=4;

        S.o.pln (a+b+c);
    }
}
```

Que Difference betⁿ post and pre decrement

	Post decrement	pre decrement
i]	decrement operator is a suffix to it's variable name	i] pre decrement operator is a prefix to it's variable name
ii]	Syntax is variable name--	ii] Syntax -- variable name
iii]	decrement the value which is present inside the variable name by 1 and update later will we using updated value e.g - a--	iii] decrement the value which is present inside the variable name by 1 and updated value will be using first e.g --a

io] Cast operator is used in

Cast operator is used in narrowing to convert larger primitive data type into smaller primitive data type.

Q.1 write a programme to calculate square of Given numbers.

→ Class square

```
{  
    public static void main (String [] args) :  
    {  
        int a = 5;  
        S.o.P.ln (a*a); //25  
    }  
}
```

Q.2 Write a programme to calculate cube of Given numbers

→ Class cube

```
{  
    public static void main (String [] args)  
    {  
        int a = 5;  
        S.o.p.ln (a*a*a); //  
    }  
}
```

Q.3 Write a programme to calculate area of rectangle

→ Class area

```
{  
    public static void main (String [] args)  
    {  
        int W = 5;  
        int L = 8;  
        S.o.P.ln (W*L);  
    }  
}
```

Que 4

WAP To calculate area of circle?

```
class Area
```

```
{
```

```
    public static void (String[] args)
```

```
{
```

```
    int a = 3;
```

```
    double pi = 3.14;
```

```
    S.o.P.println (pi * a * a);    // 28.25999999
```

```
}
```

```
}
```

Q.5

WAP To Find biggest of two numbers

```
class Bigg
```

```
{
```

```
    public static void (String[] args)
```

```
{
```

```
    int a = 15;
```

```
    int b = 25;
```

```
    S.o.P.println (a > b);    // False
```

```
    S.o.P.println (a < b);    // True
```

```
}
```

```
}
```

Q.5

WAP To calculate area of square

3] Logical operator - 1] Logical operator it is an Binary operator

2] It takes 2 values at a time.

3] Logical operator it is classified into 3 types.

1] AND (&)

2] OR (||)

3] Not (!)

1] AND
truth table

I/P1	I/P2	O/P
True	True	True
True	False	False
False	True	False
False	False	False

2] OR

truth table

I/P1	I/P2	O/P
True	True	True
True	False	True
False	True	True
False	False	False

①

class PG

{

public static void main(String[] args)

{ //AND & :

S.o.Pln (true & true); // true

S.o.Pln (false & true); // false

S.o.Pln (true & false); // false

S.o.Pln (false & false); // false

//OR || :

S.o.Pln (true || true); // true

S.o.Pln (true || false); // true

S.o.Pln (false || true); // true

S.o.Pln (false || false); // false

}

}

3] NOT (!) - Not operator it is an unary operator it take only one input at a time.

2] Not operator belongs to logical operator

3] Since majority of logical operators takes 2 values as an input hence logical operator belongs to Binary operator.

Truth table

— only one I/P so unary

I/P	O/P
True	False
False	True

• used for inverting a boolean value.

(2)

```
class P10
```

```
{
```

```
    public static void main(String[] args)
```

```
    {        // NOT !
```

```
        S.o.P10(!true);
```

```
        // False
```

```
    }
```

```
}
```

* Ternary Operator - / Conditional operator.

Ternary operator it is operator which takes 3 values at a time is called as ternary operator

It is also called as conditional operator

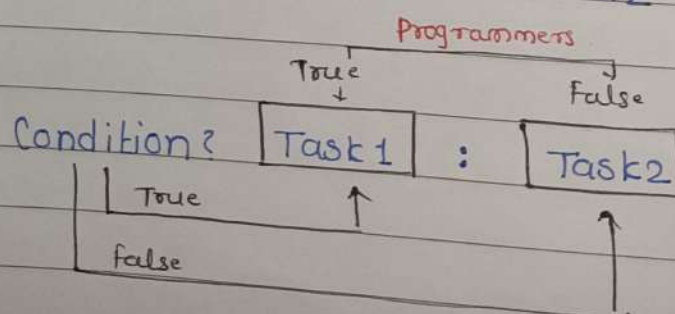
Conditional operator -

It is an ternary operators

Conditional operator takes 3 value at a time

Syntax -

Condition ? Task1 : Task2



only these can we write

Task - {
 Value
 Variable-name
 Condition

condition $\Rightarrow$ o/pTrue $\rightarrow$ Task 2False $\rightarrow$ Task 3

1] IF the condition is true the control will transfer for task 1

2] IF the condition is false the control will transfer for task 2

Que WAP To Find maximum of two numbers using conditional operators

Class P11

{

public static void main (String [] args)

{

int a = 10;

a = 10
b = 5Task $\rightarrow$ Result

int b = 5;

int res = a > b ? a : b;

int res = a > b ? a : b;

10

(10 > 5)
True

10

Result

we can reuse becz we store the value

S.o. P. ln (res); // 10

{

{

Class P11

{ public static void main (String [] args)

{

int a = 10;

int b = 5;

// perform task $\rightarrow$ print result directly

S.o. P. ln (a > b ? a : b);

we cannot reuse the output

{

{

Que WAP To check given number is positive or negative using conditional operator.

```

class P11
{
    public static void main(String[] args)
    {
        int n = 123;
        String res = n > 0 ? "+ve" : "-ve";

        S.o.P.println(res); // +ve
    }
}

```

Que 1 design a programme to check the given number is even or odd.

Que 2 Write a programme to find minimum of two numbers

Que 3 WAP To Find maximum of three no.

Que 4 WAP To Find minimum of four numbers using conditional operator.

Que 1 Design a programme to check the given number is even or odd.
→ class ternary

```

{
    public static void main (String[] args)
    {
        int n = 44; // To get remainder
        String s = n % 2 == 0 ? "is even" : "is odd";

        S.o.P.println(s); // is even
    }
}

```


Q.2 Write a programme to Find minimum of 2 numbers.

```
→ class Min
{
    public static void main (String [] args) {
        int n = 10;
        int b = 40;

        int res = n < b ? n : b;

        S.o.pln(res);    // 10
    }
}
```

Q.3 WAP To Find maximum of three numbers.

```
→ class Max
{
    public static void main (String [] args)
    {
        int a = 3
        int b = 9
        int c = 4
        int Max = a > b ? (a > c ? a : b) : (b > c ? b : c);
        S.o.pln(Max);
    }
} // 9
```

Q.4 WAP To Find minimum of four numbers using conditional operator

```
→ class Min
{
    public static void main (String [] args)
    {
        int a = 12, b = 2, c = 3, d = 4
        int min = a < b ? a < c ? a < d ? a : b < c ? b < d ? c : d : b < c ? b < d ? c : d : c : d;

        S.o.pln(min);
    }
} // 2
```

* Decision making statements -

It is use to verifying or check the condition before performing a particular task.

- 1] IF statement
- 2] if else statement
- 3] else if ladder
- 4] nested if
- 5] switch

Same Nature

- diffⁿ nature

1] IF statement -

Syntax -

True - IF (condition) - False

→ (S1) next immediate line

S2 (skip N+1 line)

condition

Class P1

{

public static void main(String[] args)

{

if (False)

S.o.p ln ("S1");

S.o.p ln ("S2"); // S2

}

}

Class P1

{

public static void main (String[] args)

{

S.o.p ln ("leave home");

- belongs to main method

String shop = "open";

if (Shop == "open") - belongs

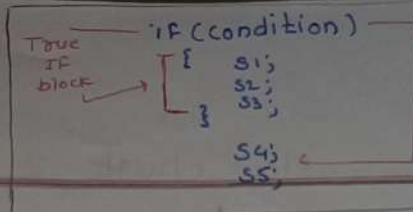
S.o.p ln ("purchase 1 kg sugar"); - belongs to if condition

S.o.p ln ("go back to home"); - belongs to main method

}

}

// all method



classmate

Date _____

Page _____

Class P1

```

{
    public static void main (Strings[] args)
    {
        System.out.println (" Leave home");

        String shop = "close";

        if ( shop == open)
        {
            // if block : get executed when condition is true
            S.o.p.ln (" purchase 1 kg sugar");
            S.o.p.ln (" purchase 1 kg salt");
            S.o.p.ln (" ice-cream");
        }

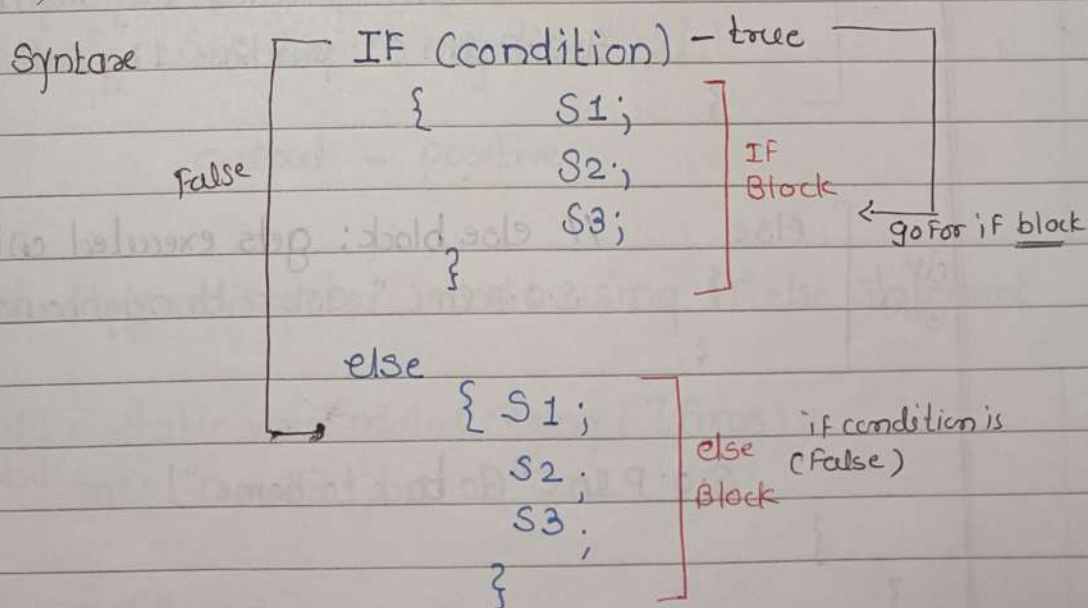
        S.o.p.ln ("go back to home");
    }
}

```

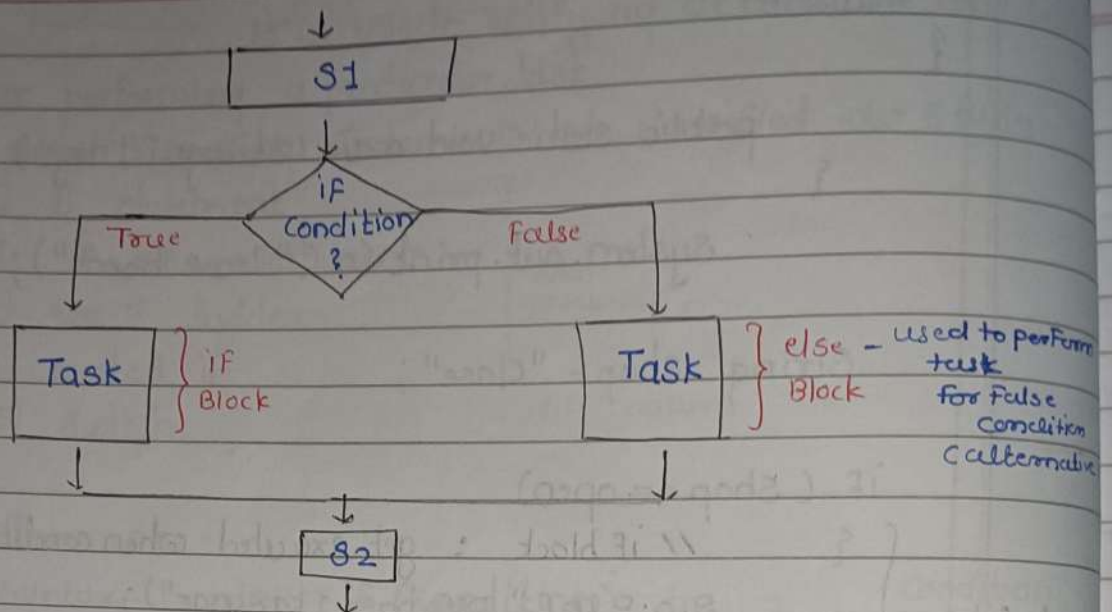
look like
single
element

output // - strings of main method becuz of if condition false so that statements not get executed.

2] if else statement -



flow chart



Class P1

```

{
    public static void main (String[] args)
    {

```

```

        S.O.P ln ("SM");
        String Shop = "open";

```

— belong to main method

```

        if (Shop == "close") {
            // if block: condition is true then gets executed
            S.O.P ln ("purchase 1 kg sugar");
            S.O.P ln ("purchase 1 kg salt");
        }

```

if Block

```

        else {
            // else block: gets executed when condition is False
            S.O.P ln ("ask with neighbours");
        }

```

else Block

```

        S.O.P ln ("Go back to home");
    }
}

```

— belongs to main method

- Q.1] WAP To check given number is positive or negative using if else statement
- 2] WAP To check given number is odd using if else statement
- 3] WAP To check minimum of 2 numbers using if else statement
- 4] WAP To check the given number is divisible by 3 and 7 using if else statement
- 5] WAP to check the given character is Vowel or not

Assignment

- Q.1 WAP to check given number is positive or negative using if else statement

```

→
class Number
{
    public static void main (String[] args)
    {
        int a = 35

        if (a > 0) (a > 0)
        {
            S.O.P.ln (" Positive"); }

        else { S.O.P.ln (" negative"); }
        }
    }
    output = positive.
  
```

- Q.2 WAP To check given number is odd using if else statement

```

class odd {
    public static void main (String[] Args)
    {
        int n = 27 ;

        if (n % 2 == 1)
        {
            S.O.P.ln ("odd Number"); }
    }
  
```

```

else
{
    S.o.pln("even number"); }
}
// odd number
}

```

Q.3 WAP to check given minimum of 2 numbers using if else statement

```

→ class Min
{
    public static void main (String [] args)
    {
        int a = 34;
        int b = 44;

        if (a < b)
        {
            S.o.pln("Min" + a);
        }
        else {
            S.o.pln("Min" + b);
        }
    }
}
// 34

```

Q.4 WAP to check the given number is divisible by 3 and 7 using if else statement.

```

→ class P1
{
    public static void main (String [] args)
    {
        int n = 39;

        if (n % 3 == 0 && n % 7 == 0)
        {
            S.o.pln("divisible");
        }
    }
}

```


// not divisible

Class Statement

```
char a = 'e';
```

```
g.o.p enc("vowel"); }
```

3

3

// vowel

as else if Ladder

Syntax -

$$\{S1\}$$
$$\{s_2;$$
$$\{ \quad 83; \}$$

Leaf (not mandatory)

go for these condition

$\% = \text{modulus}$
 For remainder of
 Number

- Q. WAP To check wheather the given number is
- 1] IF the number is divisible by 5 and 7 print "hi"
 - 2] IF the number is even and not odd no. print "bye"
 - 3] IF the number is divisible by 3 or 7 print "Good bye"
 - 4] IF none of the condition are satisfied print "Hii again"

Class Statement

```

{
    public static void main (String[] args)
    {
        int n = 12;
  
```

```

        if (n % 5 == 0 && n % 7 == 0)
        {
            S.O.P.println("hi");
        }
  
```

Condition not satisfied

```

        else if (n % 2 == 0)
        {
  
```

Condition satisfied

```

            S.O.P.println("bye");
        }
  
```

next condition consider

```

        else if (n % 3 == 0 || n % 7 == 0)
        {
            S.O.P.println("Good bye");
        }
  
```

anyone condition satisfied

करेगा नही
 IF है False तो
 then for next
 condition

```

        else
        {
            S.O.P.println("Hii again");
        }
  
```

```

    }
    // "bye"
  
```

* Note - 1] else block it is not mandatory
 2] In order to perform a particular task For False condition then only we can take the help of else block

Que. WAP To check the given character is vowel or not using else if ladder
Que. WAP To Find Maximum of 5 Numbers using else if ladder

→ Class Number

```
{ public static void main(String[] args)
```

```
{
```

```
    int a=20, b=34, c=44, d=84, f=64;
```

```
    if (a>b && a>c && a>d && a>f)
```

```
    { S.o.P.en(a); }
```

```
    else if (b>c && b>d && b>f)
```

```
    {
```

```
        S.o.P.en(b); }
```

```
    else if (c>d && c>f)
```

```
    {
```

```
        S.o.P.en(c); }
```

```
    else if (d>f)
```

```
    {
```

```
        S.o.P.en(d); }
```

```
    else
```

```
    { S.o.P.en(f); }
```

```
    }
```

```
}
```

// 84

Ques WAP to check if given character is 'vowel' or not

class P1

```
{ public static void main(String[] args)
    char a = 'k';
```

```
    if (a == 'a' || a == 'A')
```

```
    { s.o.println(a + " is a vowel"); }
```

```
    else if (a == 'e' || a == 'E')
```

```
    { s.o.println(a + " is a vowel"); }
```

```
    else if (a == 'o' || a == 'O')
```

```
    { s.o.println(a + " is a vowel"); }
```

```
    else if (a == 'u' || a == 'U')
```

```
    { s.o.println(a + " is a vowel"); }
```

```
    else if (a == 'i' || a == 'I')
```

```
    { s.o.println(a + " is a vowel"); }
```

```
    else { s.o.println(a + " consonant"); }
```

```
    }
}
```

4] Nested IF condition -

an if condition inside another if condition is called nested if condition.

Syntax -

IF (condition)

```
{
    if (condition)
    {
        statement;
    }
    else
    {
        statement;
    }
}
```

else

```
{
```

```
    statement;
```

```
}
```

Condition true

↓
Then go inside
IF block.

Que. WAP To check given character is vowel or not if character is uppercase.

→ class P4

```
{ public static void main(String[] args)
```

```
{
```

```
    char ch = 'A';
```

```
    if (ch >= 'A' && ch <= 'Z')
```

Condition For Finding character is in uppercase or not

```
    {
```

```
        // condition for vowel
```

Condition true then

it will go inside if block

```
        if (ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U')
```

```
        { s.o.p.println("character is vowel:" + ch); }
```

```
    else
```

```
    { s.o.p.println("character is not a vowel:" + ch); }
```

```
    }
```

```
    else
```

```
    {
```

```
        s.o.p.println("character is not in uppercase"); }
```

```
    }
```

```
}
```

Que. WAP To check the given number is even or odd if the number is positive

→ class Number

```
{ public static void main (String[] args)
```

```
{
```

```
    int n = 64;
```

```
    if (n > 0)
```

```
    {
```

```
        if (n % 2 == 0)
```

```
        { s.o.p.println("even"); }
```

```

else if (n % 2 != 0)
{
    s.o.p.ln("odd"); }
}

```

```

else
{
    s.o.p.ln("number is negative odd"); }

```

5] Switch -

1] Switch it is decision making statement in java. inside switch we cannot pass condition

2] Inside switch we can pass expression or literals used to achieving grouping

Syntax -

```

Switch (expression or
        literals)
{
    Case value : { S1;
                  } (multiple case values)
    Case value : { S2;
                  }
    Case value : { S3;
                  }
    Default : { S4;
               }
}

```

3] if None of the cases are satisfies, default block gets executed. (default block nature is similar to else block. These block is not mandatory)

4] the output of an expression is a value (values means literals)

Condition output is true or false
expressions gives values (Literals)

Condition = question along with
answer

$n \% 2 == 0$

default block behaves like else block

Que WAP to check the given no is even or odd using switch

Class Switch

```
{ public static void main (String[] args)
```

```
{
```

```
int n = 3;
```

```
Switch (n%2)
```

```
{
```

```
Case 0: {
```

```
S.o.P.ln("even"); }
```

```
Case 1: {
```

```
S.o.P.ln("odd"); }
```

```
}
```

```
}
```

```
}
```

Class Switch

```
{ public static void main (String[] args)
```

```
{
```

```
int n = 2;
```

```
Switch (n%2)
```

```
{
```

```
Case 0: { S.o.P.ln("even");
```

```
break;
```

```
}
```

```
Case 1: { S.o.P.ln("odd");
```

```
}
```

```
}
```

```
}
```

```
}
```

Without
Break;

even
odd

Output

With
break

even only

(break is imp otherwise it will
execute next case also)

Que WAP To check the given character is vowel or not using switch
Que WAP to check the given character is vowel or not only if the character is in lowercase switch

Que WAP to check the given character is vowel or not using switch

```
→ Class Switch1
{
    public static void main (String[] args)
    {
        char ch = 'o';

        Switch (ch)
        {
            Case 'a' : { S.o.P.ln ("a" it is a vowel" + ch); } Break;
            Case 'e' : { S.o.P.ln ("e" it is a vowel" + ch); } Break;
            Case 'o' : { S.o.P.ln ("o" it is a vowel" + ch); } Break;
            Case 'u' : { S.o.P.ln ("u" it is a vowel" + ch); } Break;
            Case 'i' : { S.o.P.ln ("i" it is a vowel" + ch); } Break;
            Default : { S.o.P.ln ("it's not vowel" ); } Break;
        }
    }
}
```

// @ it's a vowel : o

Que WAP to check the given character is vowel or not only if the character is in Lower case switch.

```
→ Class Switch1
{
    public static void main (String[] args)
    {
        char ch = 'K';

        if ( ch > 'a' && ch < 'z' )
        {
```


classmate
Date _____
Page _____

classmate
Date _____
Page _____

switch (ch)

```
{ case 'a' : { s.o.p.ln ("a"); }  
  case 'e' : { s.o.p.ln ("e"); }  
  case 'u' : { s.o.p.ln ("u"); }  
  case 'o' : { s.o.p.ln ("o"); }  
  case 'i' : { s.o.p.ln ("i"); }  
  Default : { s.o.p.ln ("it's not vowel"); }  
}
```

else

```
{ s.o.p.ln ("character is in uppercase"); }  
  // character is in uppercase
```

```
}
```

* Looping statement

1] Looping statement it is used to verify a condition multiple times.

2] Looping statement is classified into 3 types

1] While loop

2] do while loop

3] for loop

o] If condition is true then

it will continue in loop

if false then go outside from loop

otherwise gives infinity

* 1] While loop -

three things in while loop

1] condition - is imp

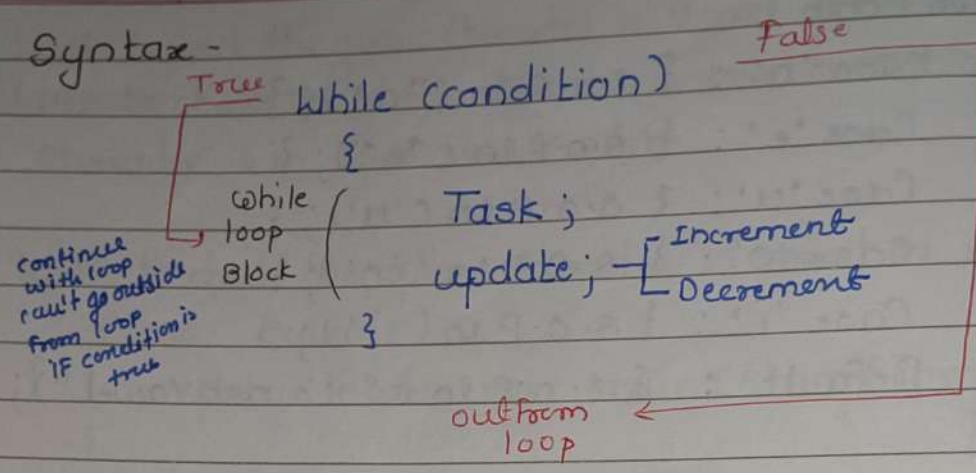
2] Task

3] update

condition is imp

for task and update

Syntax -



Q.

Print \$ for 3 times

→

class P1

```

{
    public static void main (strings[] args)
    {
  
```

```

        int n = 0;
  
```

```

        int m = 3;
  
```

```

        S.o.Pln ("main method begin");
  
```

```

        while (n < m) // condition
        {
  
```

```

            S.o.Pln("$"); // task
  
```

```

            n++; // update
        }
  
```

```

        S.o.Pln ("main method end");
    }
}
  
```

$0 < 3 = T$
 $1 < 3 = T$
 $2 < 3 = T$
 $3 < 3 = F$

Ques



Write a programme to print all the even no. between 0 to 10

Class P2

```

{
    public static void main (String [] args)
    {
        int n = 0;
        int m = 10;

        While (n <= m)    // condition
        {
            if (n % 2 == 0)    // task
            {
                S.o.P.println(n);
            }

            n++;    // update
        }
    }
}

```

* 2]

do while -

syntax

1] Task;

2] update ;

3] condition] → 2nd

1st indo

First do task
and update
and then go for
condition.

Syntax -

```

do
{
    Task;
    update;
}
While (condition);

```

print

* → 3 times

a = 0 b = 3

* do while loop performs task
even if the condition is
false for one time

inside main method
all gets executed line
by line

classmate

Date _____
Page _____

Class P3

```
{ public static void main(String[] Args)
```

```
{ int a=0;
```

```
  int b=3;
```

```
  do
```

```
  { system.out.println("*"); // task  
    a++; // update
```

```
  }
```

```
  while (a<b); // condition
```

do while loop

```
}
```

```
}
```

main method heat

line
by
line

1 → ① int a = 0

→ ② int b = 3

→ ③

do

{ s.op(*)

a++;

}

while (condition)

For stop
these loop
condition
should be
false

Note- do while loop performs task even if the condition is false for one time.

Class P3

```
{ public static void main (String[] Args)
```

```
{ int a=0;
```

```
  int b=3;
```

```
  do
```

```
  { s.o.println("*")
```

```
    a++;
```

```
  } while (a<b); // condition is false
```

```
}
```

```
}
```

so output '*' one time.

using while loop and do while also.

Que 1 WAP To print First 5 characters

Que 2 WAP To print all the no. which are divisible by 3 & 7 between 0 to 100

Que 3 WAP To print First 10 numbers in reverse order

Que 4 WAP To count all the even numbers between 0 to 150

Que 5 WAP To count all the numbers which are divisible by 3 and 9 between 0 to 100

Que 6 WAP To find the addition (sum) of first 10 numbers

Que 7

Q 1, 2, 3 Class P1

```
{
    public static void main (String[] args)
    {
```

```
        char a = 'a';
```

// Question no. 1

```
        char b = 'z';
```

```
        while (a < b)
```

```
        {
```

```
            S.o.pln(a);
```

```
            a++; } }
```

```
        int c = 1;
```

// Question no. 2

```
        int d = 10;
```

```
        while (c < d)
```

```
        {
```

```
            S.o.pln(d);
```

```
            d--; } }
```

```
        int e = 0;
```

```
        int f = 100;
```

```
        while (e < 100)
```

```
        { if (e % 3 == 0 & e % 7 == 0)
```

```
            { S.o.pln(e); }
```

```
            e++; } } }
```

Q. 4, 5, 6 Count -

Q. 4

class P1

```
{ public static void main (String [] args)
```

```
{ int n=1;
```

```
int m=50;
```

```
int count=0;
```

```
while(n<m) // condition
```

```
{
```

```
if(n%2==0)
```

```
{
```

```
count++;
```

```
}
```

task

```
n++; // update
```

```
}
```

```
S.o.pln(count);
```

```
}
```

```
}
```

Q. 6

class P1

```
{ public static void main (String [] args)
```

```
{ int a=0;
```

```
int b=5;
```

```
int sum=0;
```

```
while (a<b)
```

```
{
```

```
sum=sum+a;
```

```
a++;
```

```
}
```

```
S.o.pln(sum);
```

```
}
```


For - initialization is not mandatory if value is given
 For loop - From outside

Date _____
 Page _____

* 3] For

Looping statement-

While

do while

For

1] condition - start

1] Task

1] Initialization

2] Task;

2] update

Condition - middle

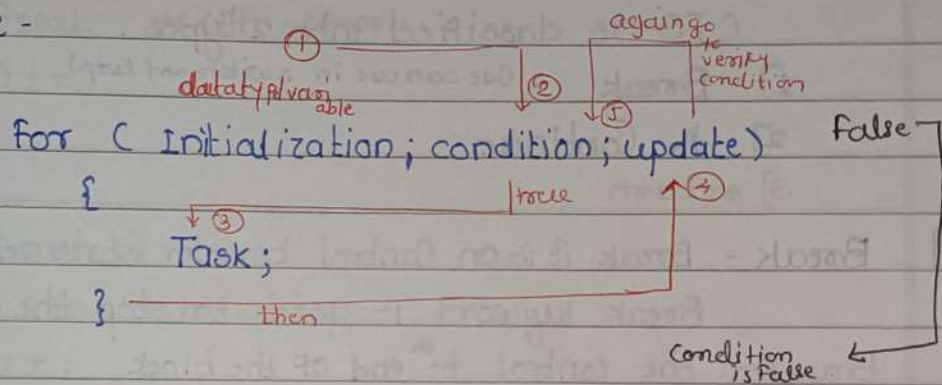
3] update;

3] condition - end

update

• do task For false condition

Syntax -



class P5

```

{ public static void main (String[] args)
{

```

```

    for (int sp = 0; sp < 5; sp++)
    {

```

```

        {

```

```

            S.O.P ln ("star");

```

```

        }

```

```

    }

```

```

}

```

output

Star
 Star
 Star
 Star
 Star

Q.1 WAP to Find product of first 5 numbers

Q.2 WAP to Find summation of all the numbers which divisible by 4 and 8 between 0 to 100.

Q.3 WAP to Find summation of first five characters

Q.4 WAP to print first five characters in Reverse order

Q.5 WAP to Find product of all the numbers which are divisible by 3 and 7 between 0 to 50

- Q.6 WAP to Find Factorial of 5
- Q.7 WAP to count the numbers of vowels (lowercase 'a' to 'z')
- Q.8 WAP to convert first 5 characters into integer format
- Q.9

* Control transfer Statement

Control transfer statement it is used to transfer the control out of the block

CTS is classified into 2 types.

- 1] Break - (We can use in switch and loop)
- 2] Continue
- 3] return

1] Break - Break it is an Control transfer statement

Break keyword it used to stop the execution and transfer the control to the end of the block

Class P1

```

{
    public static void main (String [] args)
    {
        int a = 0;

```

```

        while (a < 9)
        {
            s.o.p.ln(a);
            if (a == 1)
            {
                s.o.p.ln("if block");
                a++; break;
            }
            s.o.p.ln(a);
            a++;
        }
    }
}

```

becoz of break keyword go outside from while loop

While loop

(while loop having dependency on if block)

Break - going downwards

We can't use break keyword inside the main method
break keyword stops particular task
like in particular loop or if block

Continue - pause the execution and go for starting point
like Restart

classmate

Date

Page

Note - We cannot use break keyword in between the block.

(We get Compile time error called unreachable statement)

so we do

In order to use break keyword it should be always
defined at the end of the block

```
if (a == 1)
```

```
{ s.o.p.println("IF block");
```

```
break; // CTE error
```

```
a++;
```

```
}
```

```
if (a == 1)
```

```
{ s.o.p.println("IF block");
```

```
a++;
```

```
break; // works
```

```
}
```

2] **Continue** - ^{1]}Continue keyword it is a control transfer
statement. 2] Continue keyword it is used to transfer
the control to the beginning of loop condition

Class Ps

```
public static void main (String [] Args)
```

```
{
```

```
int a = 0
```

```
while (a < 4)
```

```
{ s.o.p.println(a);
```

```
if (a == 1)
```

```
{ s.o.p.println("if block");
```

```
a++;
```

```
continue;
```

```
}
```

pause the programme

```

s.o.penc(a);
a++;
}
s.o.penc("main end")
}
}

```

output

0
0
if block
2
2
3
3

Note - 1] We cannot define continue keyword in between the blocks (We get an compile time error for unreachable statement)

2] Continue keyword it should be always defined at the end of block.

```

if (a == 1)
{
s.o.penc("IF block");
continue; // CTE
a++;
}

```

```

if (a == 1)
{
s.o.penc("IF block");
a++;
continue; // Works
}

```

Que Can we define break and continue keyword together inside a single block. *break and continue keyword we can use in switch and looping statement only.*

Que Can we define break and continue keyword together inside a single programme.

Assignment on FOR loop

classmate

Date _____

Page _____

Q.1

WAP To print first five character in Reverse order.

Class practice

```
{  
    public static void main (String[] args)  
    {  
        for (char a = 'e'; a >= 'a'; a--)  
            { S.O.P ln (a); }  
    }  
}
```

Initialization condition update
task

Q.2

WAP To count the numbers of vowels (lowercase 'a' to 'z')

Class practice

```
{  
    public static void main (String[] args)  
    {  
        int count = 0;  
        for (char a = 'a', b = 'z'; a < b; a++)  
        {  
            if (a == 'a' || a == 'e' || a == 'o' || a == 'u' || a == 'i')  
            {  
                count++;  
            }  
            a++;  
        }  
        S.O.P ln (count);  
    }  
}
```

if block
belongs to for block
belongs to main method

Q.3

Convert first 5 characters into integer format.

Class practice

```
{  
    public static void main (String[] args)  
    {  
        int h = 0;
```

```

    For (char a = 'a' ; a <= 'e' ; a++)
    {
        h = a ;
        s.o.p.en (h) ;
    }
}

```

Q. 1 WAP To Find product of first 5 numbers

→ class practice

```

{
    public static void main (String [] args)
    {
        int product = 1 ;

```

```

        For (int a = 1 ; a <= 5 ; a++)
        {
            product = product * a ;
            s.o.p.en (product) ;
        }
    }
}

```

Q. 2 WAP To find summation of all the numbers which divide by 4 and 8 between 0 to 100.

→ class practice

```

{
    public static void main (String [] args)
    {
        int sum = 0 ;

```

```

        For (int a = 0 , b = 100 ; a < b ; a++)
        {
            if (a % 4 == 0 && a % 8 == 0)
            {
                sum = sum + a ;
            }
        }
    }
}

```



```

        S.o.p.en (sum);
    }
}

```

Q.3 WAP To Find summation of first five characters
→ class practice

```

{
    public static void main (String [] args)
    {
        int sum = 0;

```

```

        for (char a = 'a'; a < 'e'; a++)
        {
            sum = sum + a;
        }

```

```

        S.o.p.en (sum);
    }
}

```

Q.5 WAP To find product of all the numbers which are divisible by 3 and 7 between 0 to 50.

→ class practice

```

{
    public static void main (String [] args)
    {
        int product = 1;

```

```

        for (int a = 1; a <= 50; a++)
        {
            if (a % 3 == 0 && a % 7 == 0)
            {
                product = product * a;
            }
        }

```

```

        S.o.p.en (product);
    }
}

```

Q.6 WAP To Find Factorial of 5

→ class practice

```
{  
    public static void main (String [] args)  
    {  
        int fact = 1;  
        for (int a = 1, b = 5; a <= b; b --)  
        {  
            fact = fact * b;  
        }  
        S.O.P.en (fact);  
    }  
}
```

Methods

• **Modifier** - access type of method - It specifies the visibility of the method.

- 1] public - method is accessible by all classes in our application.
- 2] private - the method is accessible only in the classes in which it is defined.
- 3] protected - the method is accessible within the same package or subclasses in a different package.
- 4] Default - When we do not use any access specifier in the method declaration, java uses default access specifier by default. It is visible only from the same package only.

To create method - 1) Method header
2) Method body/block (inside set of instructions) classmate

Actual Methods in JAVA

Methods

1) It is set of instruction to perform a particular task. We can create a method with the help of method header.

1) Method header -

Modifier - modification/we can change // public

Return type - it returns PRT, object, collection, // void

Method name - // main

(Formal arguments) - parameter (datatype, variable name) // (String[] args)

Modifier return type Method name (Formal arguments)
 ↓ ↓ ↓
 public static void main (String[] args)

2) We can create a method inside class block

3) We can create more than one method inside the class block.

class P7

{

Method
1

```
public static void dance (String name)
{
    // Set of instructions
    System.out.println (name + ": Likes to dance");
}
```

Method
2

```
public static void main (String[] args) // method header
{
    // method block
    dance ("Jay"); // method name and formal arguments
    dance ("Akshay"); // method name and actual argument
}
```

It is imp to execute another method call from main method

}

* Same Logic diffⁿ values

CLASSMATE
Date _____
Page _____

Note - *¹ We can identify or ~~add~~ method with the help of method signature

2] method signature consists of method name and Formal Arguments. $\text{method signature} = \text{method name} + \text{formal arguments}$

3] We can create two types of methods

1] parameterized methods

2] e.g. datatype var.name

1] add (int a, int b)

2] Sub (int a, int b, int c)

3] dance (String name)

2] No Argument method.

e.g. demo(), whatsapp(), dance()

* Method naming conventions -

1] If a method name consist of single word the entire word has to be written in terms of lowercase

e.g. - demo(), dance(), example()

2] If a method name consists of more than one word

if First letter of the second word has to be written in terms of uppercase

e.g. - addSum(), createNew(), coreJava()

* Method call statement -

It is ~~error~~ process of transferring the control from calling method to called method is called as method call statement

Calling method -

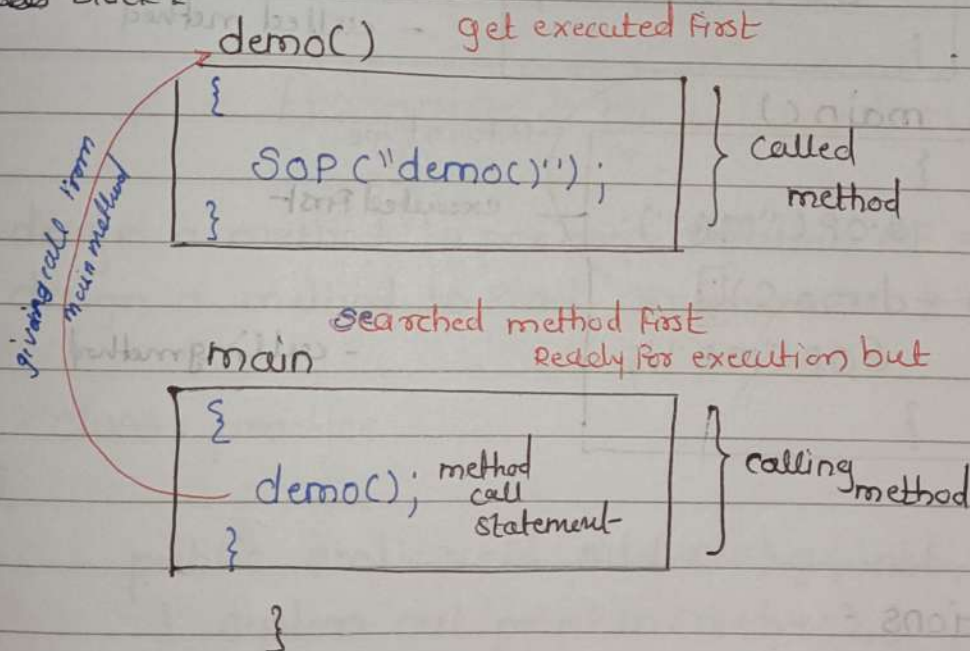
the method which tries to call another method is called as calling method.

Called method - the method which comes under execution is called as called method.

Class p1

```
{
    // called method
    public static void demo()
    {
        S.O.P.en("demo()");
    }
    // calling method
    public static void main(String[] args)
    {
        demo(); // method call statement
    }
```

~~class~~ block {



* Return type - It's a process of transferring the control from called method back to its calling method is called as return type.

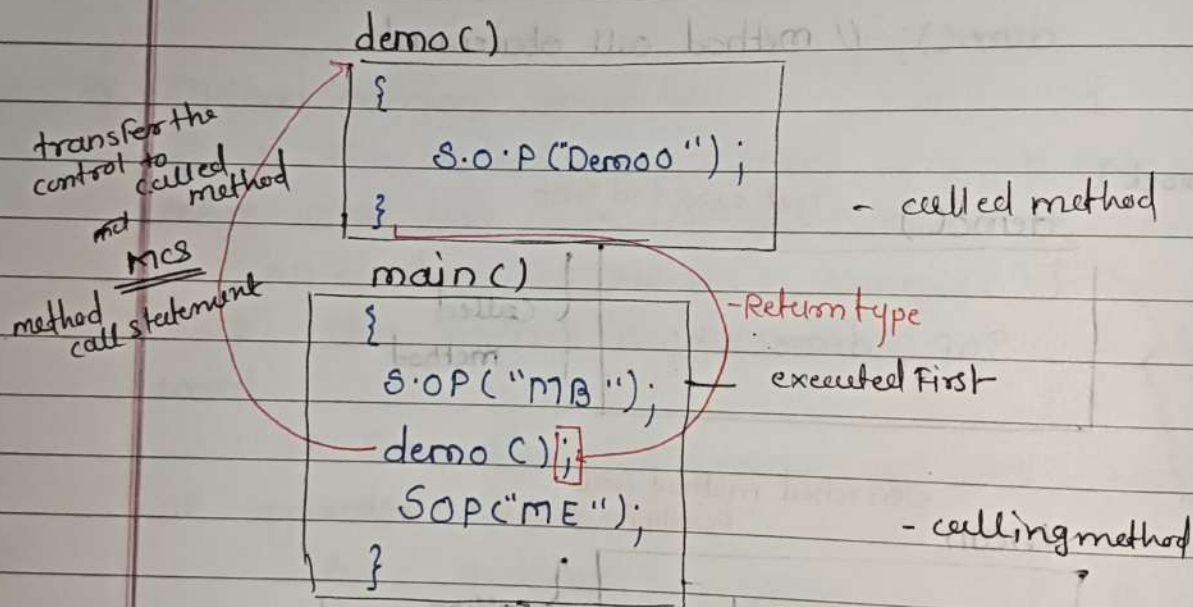
Ques P1

```

} // called method
public static void demo()
{
    S.O.P("demo()");
}

// calling method
public static void main (String[] args)
{
    S.O.P("main begin");
    demo(); // method call statement-
    S.O.P("main end");
}
}

```



Questions -

- 1] design a method to perform addition of 3 numbers. *arithmetic operator*
- 2] design a ~~number~~ ^{method} to find maximum of 3 numbers. *conditional operator*
- 3] ^{design a} to check the given no. is even or odd in string *if*
- 4] Design a method to check the given no. is positive or negative *in string*
- 5] Design a method to check the given character is uppercase or lowercase character. *else if or conditional operator*
- 6] Design a method to count all the even numbers between 0 to 10.

- 7] Design a method to check given character is vowel or not
- 8] Design a method to find summation of first five characters.
- 9] Design a method to find product of first five numbers.

```
class p1.
```

```
{ public static void demo(int a, int b, int c)
```

```
{
```

```
{ int res = a + b + c;
```

```
System.out.println(res);
```

```
}
```

```
public static void main (String[] args)
```

```
{
```

```
add (1, 2, 34);
```

```
}
```

```
}
```

parameterized
method

Assignment by different methods block

- Q.1 design a method to perform addition of 3 numbers
- Q.2 design a method to find maximum of 3 numbers

```
class practice
```

```
{
```

```
public static void add (int a, int b, int c) // method 1
```

```
{ System.out.println(a + b + c); }
```

```
public static void main (String[] args)
```

```
{ add (1, 2, 3);
```

```
max (67, 96, 47);
```

```
}
```

// main
method

Date _____
Page _____

```

public static void max (inta, intb, intc)
{
    system.out.println(a>b??a>c?a:b>c?b:c);
    // method 2
}

```

output - 6
96

- Q.3 Design a method to check given number is "even" or "odd"
 Q.4 Design a method to check given no. is "positive" or "negative"
 Q.5 Design a method to check given character is "uppercase" or "lowercase" characters.

Class practice

{

```

    public static void ide (inta) // method 1
    {
        IF (a%2 == 0)
            { system.out.println("even"); }
        system.out.println("odd"); }

```

```

    public static void posiNegative (inta)
    {
        IF (a >= 0) // method 2
            { S.O.P.println ("positive"); }
        else { S.O.P.println ("Negative"); }
    }

```

```

    public static void main (String[] args)
    {
        ide (47); ①
        charCase ('K'); ②
        posiNegative (7); ③
    }

```

// main method

output
 odd
 positive
 uppercase
 positive

```

    public static void charCase (char a) method 3
    {
        IF (a >= 'a' && a <= 'z')
            { system.out.println ("lowercase"); }
        else IF (a >= 'A' && a <= 'Z')
            { system.out.println ("uppercase"); }
    }
}

```


Q.6 Design a method to count all the even numbers between a to 10.

class practice

{

public static void count(int a, int b)

{ int count = 0; // variable // method-1

while (a < b)

{ if (a % 2 == 0)

{ count++; }

a++; }

System.out.println(count); } // printing statement

public static void main (String[] args)

{ count(0, 10);

add('t', '5'); }

// main method

public static void add (char a, char b)

{ int c = a + b;

S.o.p.println(c); }

// extra for understanding

Q.7 Design a method to check given character is vowel or not

class practice

{

public static void char1(char a)

{ if (a == 'a' || a == 'e' || a == 'u' || a == 'i' || a == 'o')

{ S.o.p.println(a + ": lowercase and vowel"); }

else if (a == 'A' || a == 'E' || a == 'U' || a == 'I' || a == 'O')

{ S.o.p.println(a + ": uppercase and vowel"); }

else { S.o.p.println(a + ": consonant"); }

// method 1

```

public static void main (String [] args)
{
    char1 ('a');
}

```

// main method

Output - a : lowercase and vowel

Q.8 Design a method to find summation of first five characters.

→ class practice

```

{
    public static void char2 (char a) // method 1
    {
        ① int sum = 0;
        ② do
            {
                sum = sum + a; task
                a++; update
            }
        while (a <= 'e'); condition
        ③ system.out.println (sum);
    }
}

```

3 elements in one method

```

public static void main (String [] args)
{

```

```

    char2 ('a');
    char3 ('a');
}

```

// main method

```

public static void main char3 (char a) // another way
{
    char b = 'b', c = 'c', d = 'd', e = 'e';
    s.o.p ln (a+b+c+d+e);
}

```

method 2

widening process happens

Output = 495
495

Q.9

Design a method to Find product of First Five numbers.

→ Class Practice

```

{
    public static void product(int a)
    {
        int product = 1;           // method 1
        do
        {
            product = product * a;
            a++;
        }
        while (a <= 5);
        S.O.P ln (product); }

    public static void main (String[] args)
    {
        product (1); }           // main method
}

```

output = 120

Ques WAP to check wheather the given character is uppercase, vowel or consonent.

Ques check wheather it is a lowercase, vowel or not
 check wheather it is a digit. digits - 0 to 9 becz character is single element
 check wheather it is a special character.

→

class P4

```

{
    public static void main (String[] args)
    {
        char ch = '$';

        if (ch >= 'A' && ch <= 'Z')
        {
            if (ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U')
            {
                S.O.P ln ("uppercase and vowel")
            }
            else
            {
                S.O.P ln ("not uppercase vowel");
            }
        }
    }
}

```

Turn -

3

$$ch = 'j'$$

```
else { s.o.p ln ("lowercase not vowel"); }
```

```

} 8.0.p1n ("It is a digit"); }

```

এদে

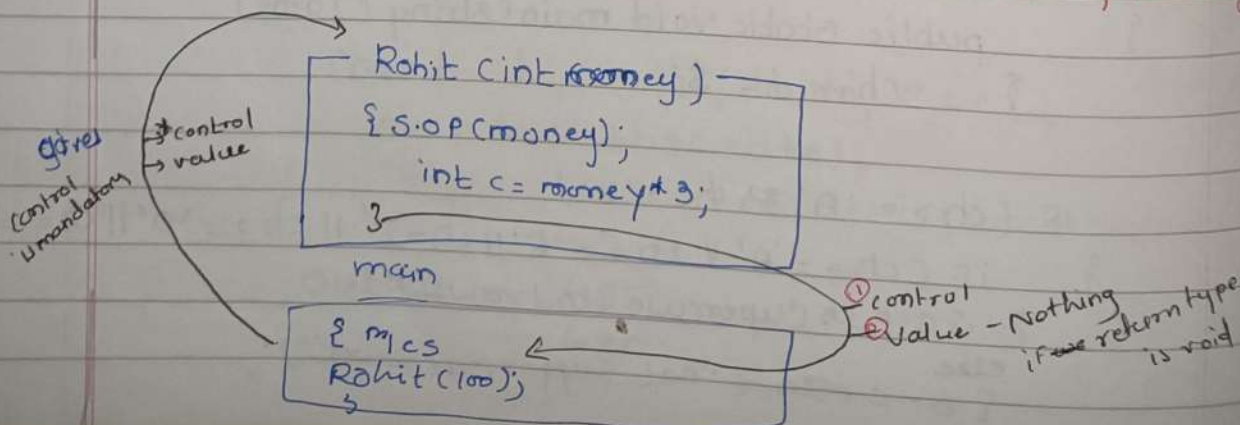
Output = It is special character

We can return 2 types of values.

- 2] Non-primitive datatype

It is a keyword in java, void means 'nothing'.
return type is void.

C but it can return transfer control by Return word only.



- ① Note - If we change the return datatype into primitive or non-primitive datatype, 'return statement' or 'return keyword' and "return value" is mandatory at end of the block.

Class P5 // storing the return value.

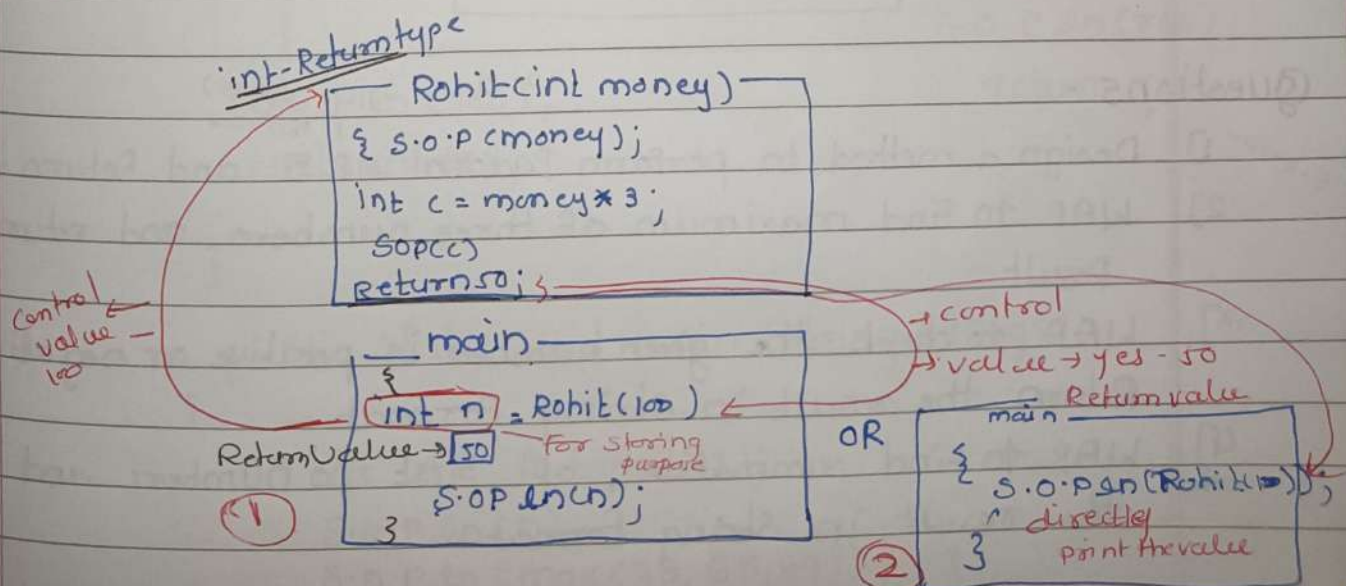
```
{
    public static int rohit (int money);
    {
        S.O.P ln (money);

        int cricket = money * 3;
        S.O.P ln (cricket);
        return 50;
    }
}
```

public static void main (String [] args)
{
 // storing the return value
 // We need to create ~~always~~ datatype and variable name
 to store return value

```
    int n = rohit (100);  
    S.O.P ln (n);  
}
```

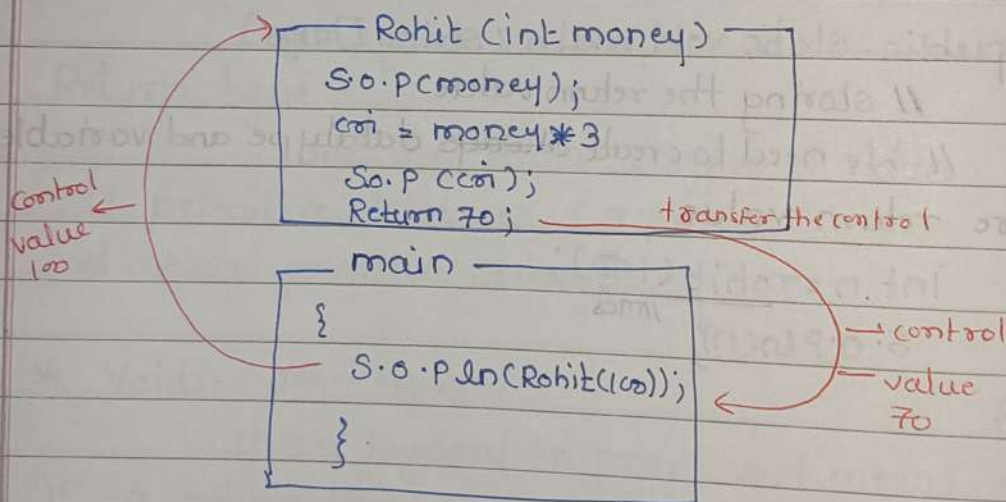
}



class P5

```
{
    public static int rohit (int money)
    {
        S.o.p.ln(money);
        int cricket = money * 3;
        S.o.p.ln (cricket);
        return 70;
    }
}
```

```
public static void main (String[] args)
{
    // printing the return value
    S.o.p.ln (rohit (100));
}
}
```



Questions -

- 1] Design a method to perform Factorial of 5 and Return the result
- 2] WAP to Find maximum of three numbers and return the Result
- 3] WAP to check the given number is positive or negative and Return the result in String format.
- 4] WAP to Find summation of First 10 numbers and Return the result in String format.

- 5] Design a method to concat first 5 characters and return the result in string format
- 6] Design a method to verify whether the given number is 'even' or 'odd' and return a Boolean condition.
- 7] Design a method to count all the even numbers between 25 to 75 and return the count.

Assignment

Q.1 Design a method to perform Factorial of 5 and Return the Result.

Q.2 WAP to Find maximum of three numbers and return the result

Class practice1

```
{
    int a=1;                                Return value
    public static int Factorial(int a)    int a=5;
    {
```

1] S.O.P ln(a);

int res=1;

while (a <= 5)

{ res = res * a;

a++;

}

S.O.P ln(res);

return res;

}

2] int res = 1;

while (a >= 1)

{ res = res * a;

a--;

}

S.O.P ln(res);

return res;

```
{
    public static void main (String [] args);
    {
```

int n = Factorial(1);

S.O.P ln(n);

S.O.P ln(max(23, 65, 89)); }

Date _____
Page _____

*return value
in the form of datatypes*

```
public static int max (inta, intb, intc)
```

```
{
    int res = a > b ? a > c ? a : b > c ? b : c;
    s.o.p ln (res);
    return res;
    return statement
}
```

Q.3 WAP to check the given number is positive or negative and Return the result in string format.

Q.4 WAP to Find summation of First 10 numbers and Return the result in string format.

→ class practice2

```
{
    public static String posiNegative (int a);
    {
        String res = "Number is ";

        if (a > 0)
        {
            String res = res + " Positive";
        }
        else if (a < 0)
        {
            res = res + " Negative";
        }
        Return res;
    }
    public static void main (String [] args)
```

main
method

```
{
    S.o.p ln (posiNegative (-5));
    S.o.p ln (sum (1));
}
```


public static String sum(int a)

{
 int sum = 1;
 String J = " sum is : " ;

 do
 {
 sum = sum * a;
 a++;
 }

 while (a <= 10);

 J = J + sum;

 Return J;

}

Q.5 Design a method to concat first 5 characters and the result in string format.

→ Class practice 3

{

 public static void main(String[] args)

 {

 S.o.pln (conchar(a));

 }

 public static String conchar(char a)

 {

 String b = " first 5 characters a: " ;

 while (a <= 'e')

 {
 b = b + a;

 a++;

 }

 Return

Return b;

}

}

- Q.6 Design a method to verify whether the Given number is even or odd and return a Boolean condition.
- Q.7 Design a method to count all the even numbers between 25 to 75 and Return the count.

→ class practice4

{

public static void main (String [] args)

{ S.o.p ln (evenOdd (34));

int c = Count (25);

S.o.p.ln (c);

}

public static Boolean evenOdd (int a)

{ Boolean b = (a % 2 == 0);

return b;

}

public static int count (int a)

{ int count = 0;

while (a <= 75)

{ if (a % 2 == 0)

{ count ++; }

a ++;

}

return count;

}

}

main

I

II

CT → javac P6.java
 verification happens
 Runtime → java P6
 execution happens

classmate

Date _____
 Page _____

IMP * Note -

1] Whenever we call method from printing statement
 (No argument method or parameterized method)
 Return type is mandatory.

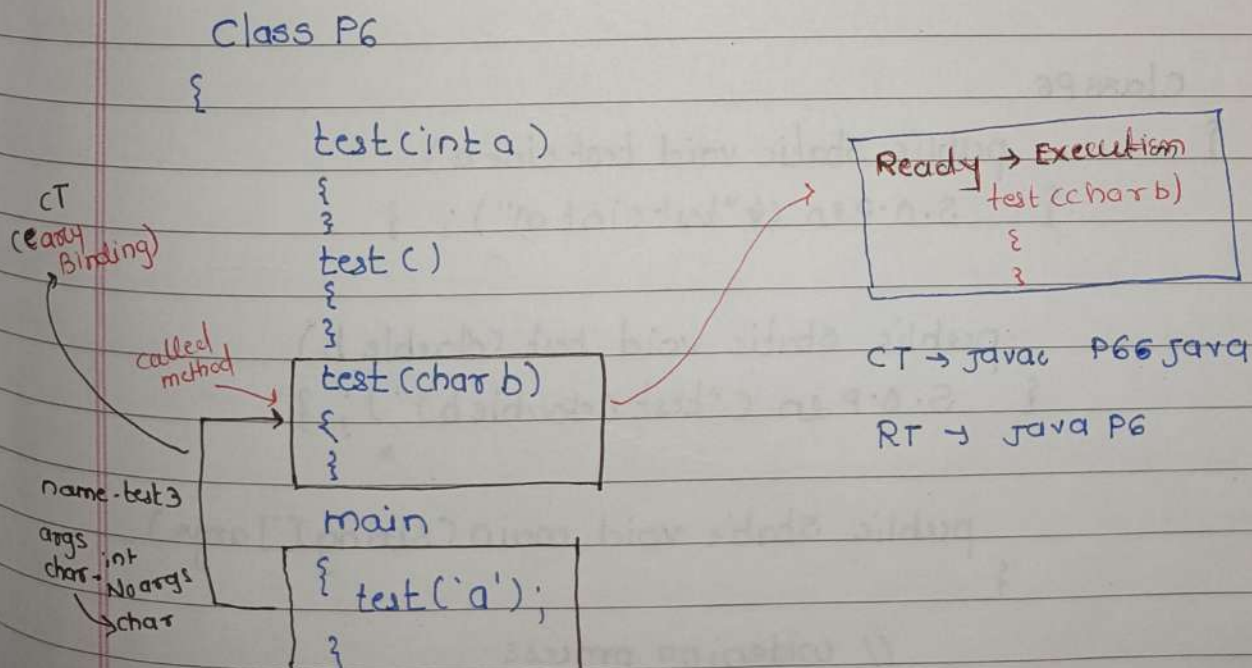
2] Whenever we call no argument method or parameterized method from a variable. Return value is mandatory.

Questions - 1] Can we declare more than one method inside class block.
 2] Can we declare more than one method inside the class block
Same name and same arguments
 Not yes (datatype should be different)

* Method Overloading -

Class consists of more than one methods with same ^{method} name and different arguments is called as method overloading.

Method overloading it is also called as early Binding



One condition more time - looping
many condition - decision making state

For storing
control etc

classmate
Date _____
Page _____

Inside the class blocks we cannot declare more than one method with same name or same arguments.

Class P6

```
{
    public static void test c)
    {
        S.O.P.println("test c");
    }

    public static void test (int a)
    {
        S.O.P.println("test (int a)");
    }

    public static void test (char b)
    {
        S.O.P.println("test (char b)");
    }

    public static void main (String[] args)
    {
        test('a');
    }
}
```

Class P6

```
{
    public static void test (int a)
    {
        S.O.P.println("test (int a)");
    }

    public static void test (double b)
    {
        S.O.P.println("test (double b)");
    }

    public static void main (String[] args)
    {
        // widening process
        // converting char --> int
        test('a'); // ascii : 97 : int
    }
}
```

so widening happens

Que 11

char → int
then double
for saving memory
wastage.

loop - by calling method

classmate

Date _____

Page _____

or directly

* Cast operator used for value not variable name

e.g. char a = (char) 13;

char a = (char) b;

↳ value present within

For narrowing.

class P6

```
{ public static void test(int a)
{
```

```
    S.O.P.println("test(int a)");
```

```
}
```

```
public static void main (String [] args)
```

```
{ // narrowing process
```

```
    test(int) (23.122); }
```

```
}
```

Que 1 Design a method to perform arithmetic operation of 2 Number and 3 Numbers.

→ class P3

```
{
```

```
    public static void
```

* Recursive method -

a method calling itself is called as recursive method.

Class P7

// recursive method

```
{  
    public static void test()  
    {
```

// called method

```
        S.o.pln("test()");
```

```
        test(); // MCS - calling itself again
```

```
    }
```

```
    public static void main (String[] args)
```

// calling

method

```
{
```

```
    test(); // MCS
```

```
}
```

1] Here in the above programme the test method it's been called itself again such that it provides an infinite loop.

2] In order to stop the infinite loop which occurs during recursive method, we need to use a condition.

3] The condition which is required to stop the recursive method is also called as base condition.

Ques- Write a programme to print first 5 no. using recursive method


```

int b = a++;
if (b > 5) {
    s.o.p(b);
    test(a);
}

if (a <= 5) {
    a++;
    s.o.p(a);
    test(a);
}

```

Que- Write a programme to print ^{5 *} First 5 no. using Recursive method.
Class P1

```

public static void test (int a)
{
    s.o.p.ln("*");
    a++;
}

```

①

```

if (a == 1)
{
    test(a);
}
else if (a == 2)
{
    test(a);
}
else if (a == 3)
{
    test(a);
}
else if (a == 4)
{
    test(a);
}

```

```

② if (a == 1)
{
    s.o.p.ln("*");
    test(a);
}
else if (a == 2)
{
    s.o.p.ln("*");
    test(a);
}
else if (a == 3)
{
    s.o.p.ln("*");
    test(a);
}
else if (a == 4)
{
    s.o.p.ln("*");
    test(a);
}

```

```

public static void main (String[] args)
{
    test(0);
    s.o.p.ln("Hello");
}

```

```

else if (a == 5)
{
    s.o.p.ln("*");
    test(a);
}

```

Output -

```

*
*
*
*
*
Hello

```

③ Base Condition -

① Parameterized method

class P1

{

public static void test(int a)

{

if (a == 5)

{

return;

}

S.o.P ln(a);

a++;

test(a);

}

public static void main (String[] args)

{

test(a);

}

}

if you take
test(a);
↓
value inside
Increment operator
never works

Variable name - We using variable name
here but it will take updated value.
So 0 becomes 1 then

Return
only
control
back to its
main
method.

Return - It will used for transfer the control from called from calling method.

- We cannot perform Recursive method and base condition inside the No argument method.

accessing - 1] For storing
2] For printing

• find default value of all datatypes

classmate

Date _____
Page _____

* Global Variables -

- 1] Global variables are declared inside the class blocks
- 2] Global variables are classified into two types.
 - 1] Static global variable
 - 2] Non-static global variable
- 3] Anywhere we can access. 4] ^{we can declare each and every more one variable but variable name has to be different (including static and non-static together).}

1] Static Global variable -

- 1] A variable prefixed with a static keyword and declared inside the class block are called as static variable.
- 2] We can declare more than one static variable inside class block but variable name has to be different.
- 3] Static variable consists of default values (default values depends on datatype). ^{Global variable consist default values}
- 4] Syntax -

Static ^{keyword} datatype Var_name = Value;

- 5] We can access static variable in two ways ^(From any block inside class block)

- 1] Directly by variable name.

- 2] with the help of class name as reference.

⊙ ⊙ and also after object creation we can also access by with the help of object name as reference or this ^(inside non-static keyword)

Class P1

{

Static int n = 10;

public static void main (String[] args)

{ // Directly by variable name
S.O.P ln(n);

// with the help of class name as reference

S.O.P ln(P1.n);

}

^{↑ work for reference}
(class name)

IMPORTANT -

We can access a variable by its variable name for what purpose?

- 1] Calling a variable name from printing statement for printing value present inside the variable.
- 2] For initializing a value. (passing value and storing value)

```
class P1
```

```
{
```

```
    static int n;
```

initialization is not mandatory - passing a value

```
    public static void main (String[] args)
```

```
    {
```

```
        // Directly by variable name
```

```
        n = 100; // initialization of static variable
```

```
        // printing the value of static
```

directly

```
        S.o.P.println(n); // variable directly
```

```
        // By class name as a reference
```

```
        P1.n = 200;
```

```
        S.o.P.println(P1.n);
```

```
    }
```

```
}
```

2 Rules - ^{use} For accessing class name a reference

① Important 1 -

We can access a static variable from one class to another class with the help of class name as reference

access - 1] storing
2] printing

Global variable - consists default values
Local variable - does not consist default values

classmate

Date _____

Page _____

① one class

```
class P1
```

```
{
```

```
    static int n = 10;
```

```
}
```

② another class

```
class P2
```

```
{
```

```
    static int a = 20;
```

```
    public static void main (String[] args)
```

```
    {
```

```
        // static variable of its own class
```

```
        S.o.p.println(P2.a);
```

```
        // static variable of another class
```

```
        S.o.p.println(P1.n); // printing
```

```
        P1.n = 50; // storing
```

```
        S.o.p.println(P1.n);
```

```
    }
```

```
}
```

Output -

20

10

50

Ques - Default values of all datatypes of static variable

→ class P1

{

```
static byte b1;  
static short b2;  
static int b3;  
static long b4;  
static char b5;  
static Boolean b6;  
static Float b7;  
static double b8;  
static String b9;
```

Global variable
default values

byte = 0

Short = 0

int = 0

long = 0

char = _

Boolean = false

Float = 0.0

double = 0.0

String = Null

{ public static void main (String[] args)

{

S.o.Pln(b1);

S.o.Pln(b2);

S.o.Pln(b3);

S.o.Pln(b4);

S.o.Pln(b5);

S.o.Pln(b6);

S.o.Pln(b7);

S.o.Pln(b8);

S.o.Pln(b9);

}

}

output

// ~~Null~~ 0

// 0

// 0

// 0

//

... blank

// false

// ~~false~~ 0.0

// 0.0

// null.

class name as reference $\rightarrow$ static variable (class block)

Static - loaded for only one time

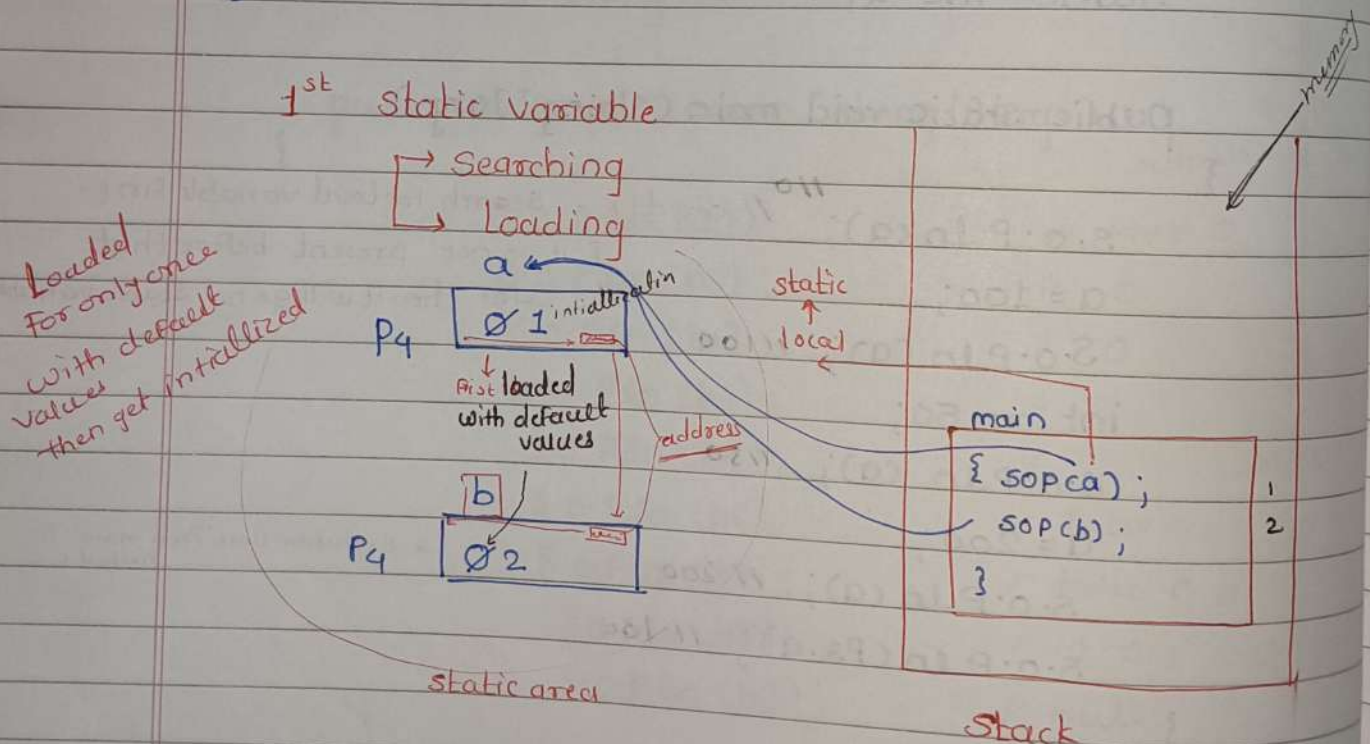
classmate

Date _____
Page _____

Searching Loading Execution

```
Class P4
{
    static int a=1;
    static int b=2;

    public static void main (String[] args)
    {
        S.o.P ln(a);
        S.o.P ln(b);
    }
}
```



Static area - static members get loaded in static area

Stack - temporary memory for execution it will clear after execution.

Stack (temporary memory for execution it will clear after execution)

* Static block -

1] A block which is prefixed with static keyword and declared inside the class block is called as static block.

Syntax -

```
    static
    {
        statements;
    }
```

2] A programmer cannot call a static block because static block does not consist any name.

3] A static block does not return any value back to programmer because it does not consists of return type.

4] We can declare more than one static block inside class block.

5] All the static block gets executed in top to bottom order before the execution of main method begins.

6] Static block always gets executed for only once.

class P5

```
{
```

```
    static // static block
```

```
    {
        S.O.P ln ("Staticblock-1");
    }
```

- gets executed first before execution of main method begins.

```
    public static void main meth (String[] args) // main method block
    {
        S.O.P ln ("main method");
    }
```

```
    static // static block
```

```
    {
        S.O.P ln ("Staticblock-2");
    }
```

```
}
```

Static block

* Static Initializer block -

* Important - static block it is also called as Static initialize block

because 1] We can initialize a static variable from Static block before the execution of main method begins.

Class P7

{
element -

① static int a;

② static
{

a = 100;

}

• static block executed
before execution of main
method begins

③ public static void main(String[] args)
{

s.o.pln(a); // 100

}

}

Questions -

1] can we access a static variable from a static block.
(How many ways)

2] can we initialize a static variable from a static block
(How many ways)

3] When it is mandatory to access a static variable with the
help of class name as a reference from a static block.

Static

Static Members -

- ① static variable
- ② static method
- ③ static Block - executed

classmate

Date _____

Page _____

top to bottom before the execution of main method begins - executed automatically without calling

Class P1

```
{
    static int a;
    {
        test('b');
    }
}
```

```
static public void test()
{
    a = 45;
    S.o.p.ln(a);
    P1.a = 35;    S.o.p.ln(P1.a)
}
}
```

```
public static void main (String[] args)
{
    test();
    S.o.p.ln(a);
    a = 35;
    S.o.p.ln(P1.a);
}
}
```

```
static public void test (char a)
{
    P1.a = a;
    S.o.p.ln(a); → local variable
    S.o.p.ln(P1.a) → static
}
}
```

Note - there are 3 components of static keyword inside class block

- 1] static variable -
 - 1] gets executed for more than once
 - 2] gets executed in two ways
 - 1] directly by variable name
 - 2] with the help of class name as reference has name
- 2] static method - get
 - 1] static method executed for more than once
 - 2] method gets executed in two ways
 - 1] directly by method name
 - 2] with help of class name as reference has name

3] static Block -

- 1] get executed for only once
- 2] does not have return type and method name
- 3] so calling is not possible no name

execution purpose
main method and execution
static block get executed first
then static method through method call statement

- * for execution method block is mandatory.
- execution - ① print a value
- ② or initialization

* Static method

• A method which is declared inside the class block and prefixed with static keyword is called as static method.

• We can declare more than static method inside the class blocks.

• We can access a static method in two ways

1] Directly by method name

2] With the help of class name as reference.

class P8

{

static public void test()

{ s.o.p.ln("Hello world");

}

public static void main(String[] args)

{

test();

P8.test();

}

}

When we can access from with the help of class name as reference

• We can access a static ^{method} variable from one class to another class with the help of class ^{name} as reference.

one class

①

class P8

{

static public void test()

{ s.o.p.ln("test");

}

}

2nd class

②

class P9

{

public static void main(String[] args)

{ //static method from another class

P8.test()

}

}

• Whichever has name get loaded first for calling.

loading sequence
1) static variable
2) static method
3) static block

But static block executed before the static method

Que - Can we access a static ^{method} from static block.
from static block we can call a static method in two ways
1) Directly by method name
2) with the help of class name as reference

```
class P9
{
    static // static block
    {
        test();
        P9.test();
    }
}
```

static block gets executed first

loaded and kept ready but not executed. execution happens after calling.

```
static public void test() // static method
{
    S.O.P.println("test");
}
```

loaded first before static block

```
public static void main(String[] args) // static method
{
}
}
```

Que - Can we access a static variable from a programmer created static method.

```
→ class P9
{
    static int a;

    public static void test()
    {
        S.O.P.println(a);
        P9.a = 200;
        S.O.P.println(P9.a);
    }
}
```



```
public static void main (String[] args)
{
```

```
    P10.test (c);
```

```
}
```

```
}
```

Ques - Can we initialize a static variable from a parameterized method (static method)

→ class P11

```
    static int a;
```

```
    public static void test (int n)
```

local variable

→ there is also static method

```
{
```

```
    P11.a = n;
```

```
    S.O.P.println (P11.a);
```

```
}
```

```
public static void main (String[] args)
```

```
{
```

```
    test (22);
```

```
    test (32);
```

```
    test (12);
```

```
    S.O.P.println ("1");
```

```
}
```

1] Searching

2] Loading

3] execution - happens only when we call

4] to get some result

loaded
in static
area

Static variable
Static method

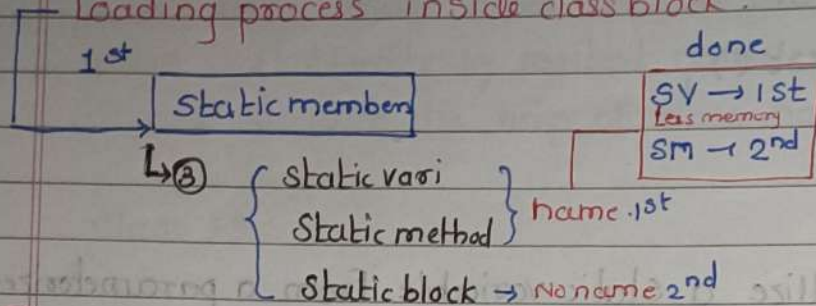
Name loading

Stack ← static block

Noname - execution

* Loading process of static members (loading process of class)

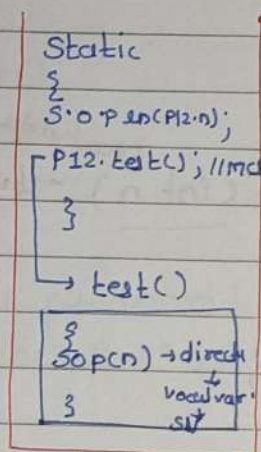
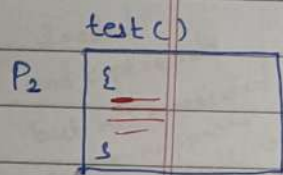
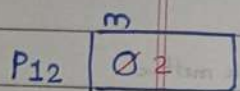
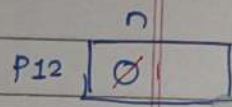
Loading process inside class block



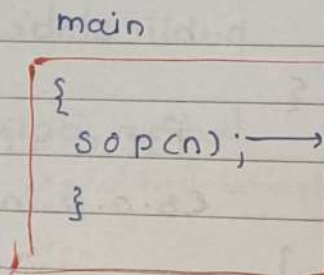
①

②

③



execution of main
method
beginning



directly
↓
local var
↓
static class block

Static
area

Stack
temporary memory
allocation for
execution purpose

Stack
everything removed
after execution

* Rule steps -

1] during the loading process of class compiler always load static variable first.

2] once after loading process of static variable is completed the compiler will load static method next

3] once after the loading process of static variable and static method is completed then the execution of static block will begin.

4] once after the execution of all the static block is complete then the execution of main method begins.


```
class P12
```

```
{    // static variable
    static int n=1;
    static int m=2;
    // static method
    public static void test()
    {    s.o.p.ln(n);
```

```
    }
    // static block
```

```
    static
    {    s.o.p.ln(P12.n);
        P12.test();
    }
```

```
    public static void main(String[] args)
```

```
    {
        s.o.p.ln(n);
    }
```

Static Disadvantage - all static members are loaded for only once
static block gets executed for only once.

get loaded for only one time - static variable and static method
get executed for only one time - static block
that's why static is constant

classmate

Date _____
Page _____

* **Non Static** - gets loaded for many times - once for each object

1] Non static members are classified into 3 types

1] non static variable

2] Non static method

3] Non static block

2] all the non-static members are not prefixed by static keywords.

3] all the non-static members are declared inside the class block.

Class P1

```
{  
    // non static variable  
    datatype variable_name = value;
```

```
    // non static method  
    modifier return_type method_name (args)  
{  
    S.O.P in ("non static method");  
}
```

```
    // Non static block
```

```
{  
    S.O.P in ("non static block");  
}
```

```
    main.method
```

```
}
```


- Main method get loaded before static block
It is also static method

- No memory allocated for NS block is more

* Note -

1] We are creating memory for entire class but we are stored only for non-static members because static members are already loaded in static area (automatically)

Object ^{It is} - block of memory - To store Non-static members for entire class.

* creation to store non-static members

We have to create an object

Step 1

It is block of memory for entire class

in that we only stored N's member because static member is already loaded

↓
In order to access N's member we need address of object

Static creating one memory - for storing address.
address - belong to object - which belongs to entire class.
to class then

Object reference Variable - It is holding the reference of object (which is meant to be stored address. (Specific) of the object)
• To generate address object creation is required.

Address of object -

datatype variable name

↓
Non-primitive datatype

↓
But we cannot write string over here

So we can write class name for identification.

already
• address is comes in string format
↓
already Generated

1] create a memory to store address

2] create a memory to store memory.

address comes in string format but we have to write class name for identification

* Object Reference Variable -

- 1] For non-static member compiler does not allocate memory automatically
- 2] In order to allocate memory for non-static members we need to create object for entire class
- 3] before creating object we need to create variable to store the address of the object first
- 4] The variable which stores the address of the object is called as object reference variable

Ques - What is object reference variable.

→ It is variable which is used or created to store the address of the house object

- 5] We need to create object reference variable for the class

* Object - reference variable -

Syntax

Class_name ref_var_name

ref_var_name - to store the address of the object which is created for the class

• Note -

For a class we can create more than one object reference variable

but object reference variable name should be different

eg - class P2

{

public static void main (String [] args)

{ P2 n;

P2 n1;

// P2 : class name

// n and n1 : object reference variable (local variable)

}

}

constructor
is most imp
① new P3()
new P3()
2 objects for same class
bt diffn address

object - get loaded one times
but there are multiple
times than classmate
to add non static members

* Constructor -

1] Constructor it is used to create an object for a class
With help of new keyword

Syntax -

Class name var_name = new class_name();

2] We can create more than one object for a class
(but object reference variable name has to be different)

class P3

{

public static void main (String [] args)

{

P3 n = new P3();

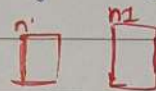
P3 n1 = new P3();

S.o.p ln (n);

S.o.p ln (n1);

}

}



object creation

// address of the object

// (two diffⁿ object generated
diffⁿ address for same class)

new - create new block memory

(.) constructor is imp - will make
sure object belongs to that class

main (under execution)

```

{
  OC -> P3 n = new P3();
         |
         v
         P3@100
         S.o.p (n); // P3@100

         P3 n1 = new P3();
         |
         v
         P3@101
         S.o.p (n1); // P3@101
}
  
```

address
of the
object

P3@100 - address of the object (P3)

block of
memory

1 new

P3@200

HEAP
AREA

HEAP

Area - we can create more than one object
in heap area to local non-static member

• Result of main method belongs to it's own class.

Important 1 -

1] From a class we can create an object for its own class and for another class. (We can create more than one object) own class and another class

```
class P4
{
    public static void main(String[] args)
    {
        P4 n = new P4();           // it's own class
        P3 n1 = new P3();          // another class
        S.o.P.en(n)
        S.o.P.en(n1)
    }
}
```

* Non-Static Members

① Non-Static variable -

1] A variable which is not prefixed with static keyword and declared inside the class block is called as non-static variable.

2] We can declare more than one non-static variable inside the class block but variable name has to be different.

3] To load non-static variable object creation for class is mandatory.

4] We can access a non-static variable with the help of object reference variable.

5] Non static variable consists of default value.

access N-S variable
inside static
method

① With the help of
Object reference
variable

access N-S variable from
constructor, N-S member

① Directly - when there is no local
variable and static variable
with same name

② By using this keyword
points to current object search first for
N-Static member

class P4

{ int n;

public static void main (String [] args)

{

P4 obj = new P4();

considered as local variable

S.O.P.println(obj);

// address of the object

S.O.P.println(obj.n)

// default value of non-static variable

obj.n = 20;

// initialization of ns variable

S.O.P.println(obj.n);

// updated value of ns variable

}

}

Important 2 -

Non-static variable gets loaded only once for each object creation.

class P4

{ int n;

- same nonstatic variable get loaded for more than one

public static void main (String [] args)

once for each object

{

P4 obj = new P4();

S.O.P.println(obj);

S.O.P.println(obj.n)

obj.n = 20;

S.O.P.println(obj.n);

①

P4 obj1 = new P4();

S.O.P.println(obj1);

S.O.P.println(obj1.n)

obj1.n = 20;

S.O.P.println(obj1.n);

②

}

}

Questions -

- 1] Write a difference between static variable and non-static variable.
- 2] Write a difference between local variable and non-local variable.
- 3] What is new keyword "java"?

programming question -

- 1] WAP to check the given number is prime number or not (between)

- 2] WAP to count all the prime number between 0 to 100

Q.1

Static Variable

Non-static variable

- | | |
|---|--|
| 1] Static variable is prefixed with static keyword. | 1] Non-static variable is not prefixed with static keyword. |
| 2] Static variable <u>loaded for only once</u> . | 2] Non-static variable loaded for more than one ^{but once} <u>for each object</u> . |
| 3] Static variable get loaded automatically in <u>Static area</u> . | 3] Non-static variable does not get loaded automatically. |
| 4] To store static variable object creation is not required. Compiler create block of memory for them. | 4] <u>Object</u> To store Non-static variable <u>object creation</u> is mandatory. |
| 5] We can access static variable in two ways. <ol style="list-style-type: none"> 1] Directly by variable 2] With the help of class name as reference. | 5] We can access Non-static variable by only with the help of <u>object reference variable</u> (store address of the object) as reference. |
| 6] Static variable gets loaded in <u>Static area</u> . | 6] <u>Non static variable</u> loaded in <u>object</u> which is in <u>HEAP area</u> . |

Non-static variable inside the object
and object inside the heap area

8/2 Local Variable

Non-Static variable

1] Local variable declared inside method block.

2] We cannot access local variable outside of method block.

3] Local variable does not consist default values.

4] In order to access local variable initialization is mandatory. ^{we need to} before initialization declaration of datatype and variable name.

5] We can access local variable directly by variable name.

6] Local variable get automatically loaded. object creation is not required.

7] Local variable executed through main method from stack (temporary memory allocation)

1] Non-static variable declared inside class block.

2] We ^{can} ~~cannot~~ access non-static variable from anywhere.

3] Non-static variable consists default values.

4] In order to access non-static initialization is ^{not} mandatory (passing value) becz it consist default values.

5] We can access non-static variable with help of object reference variable ^{name} as reference.

6] Loading of non variable object creation is mandatory.

7] Non static loaded in ^{object} ~~heap~~ which present in ~~HEAP~~ ^{area}

it is object

- it's own class
and another class

Constructor use new keyword
new - 1

HEAP ^{area for} → object ^{get located} → ^{classmate} inside object
Non-static member

- in single block we cannot have same variable name.
- address receive anyone (with relation) receiving → concentration on object reference variable.

```
class A
{
    int a = 10;
}
```

```
class B
{
    int a = 20;
    public static void main (String[] args)
    {
```

ग्रन्थ class मध्ये non-static (दुसऱ्या class) access करायचे आहेत तिथे object creation करावे लागते / (दुसऱ्या class चे) or object reference variable

we can create object for two people

① For another class

```
    {
        A obj = new A(); // object creation for class A from class B
        S.o.pln(obj); // address of class A
        S.o.pln(obj.A); // non-static from class A // 10
    }
```

② it's own class

```
    {
        B obj1 = new B(); // ob for class B
        S.o.pln(obj1) // address - B
        S.o.pln(obj.B); // value from - B // 20
    }
```

* Constructor - It used

only one difference b/w constructor and method
only change is constructor does not have return type

Constructors are classified into two types.

- 1] No argument constructor
- 2] parameterized constructor.

constructor It is used to create block of memory with the help of new keyword for class

inside class - 8 blocks

Static
① Static variable
② Static method
③ Static block

Non-Static
④ NS-variable
⑤ NS-method
⑥ NS-Block
⑦ constructor

IMP - object Reference variable
is a local variable
so initialization is mandatory
It cannot access outside of the method block

1] No argument constructor -

the constructor which does not takes any value called as No argument constructor.

```
class B
{
    B()
    {
        S.O.P.println("B()");
    }

    public static void main (String[] args)
    {
        B obj1 = new B();
        // B()
        // B: class name
    }
}
```

• For execution of constructor
object creation is required

2] parametrized constructor -

the constructor which take consists of parameters (datatype, variable name) such constructors are called as parameterized constructor.

```
class B
{
    B(int a, int b) // a, b: local variable
    {
        S.O.P.println(a); // directly: 1st: local variable
        S.O.P.println(b);
    }
}
```

During the object creation -

Non-Static Block
Data gets executed
Page

Constructor - constructor will load static member and non-static members
two things -
- deciding execution
- all are get loaded

```
public static void main (String[] args)
{
    B obj1 = new B(1,2);

    // B()
    // B(class_name)
}
}
```

- **Important** - We can declare both the type of constructors inside the class block. (parameterized and no argument constructor)
- constructor is similar to method (parameterized and no argument) totally only difference is constructor does not consist any return type. (99.5% similar) and also default constructors present

* Constructor overloading.

class consists of more than one constructor with same class name and different arguments is called as constructor overloading.

```
class P5
{
    P5() // no arg constructor
    {
        S.o.P.in ("P5()-1");
    }

    P5(int a) // parameterized constructor
    {
        S.o.P.in ("P5(int): "+ a);
    }
}
```

• Default constructor is present
no argument default constructor

a: local variable

* IF ^{we} want to address of object

Constructor execution is mandatory

classmate

Date _____
Page _____

```
public static void main (String[] args)
```

```
{
```

```
    P5 obj = new P5('a');
```

// ascii value of 'a' is 97, int
widening happens)

```
}
```

```
}
```

Note - Inside the class we cannot define more than one constructor with same class name and same argument (we get compile time error)

Default constructor (No argument type) -

When a programmer fails to define atleast one constructor inside the class block (No argument and parameterized constructor).

The compiler automatically adds default constructor inside the class block of no argument type.

Note - In java there is no concept called as default parameterized constructor. (because compiler does not give variable name (concept) to parameterized constructor).

```
class P6
```

```
{
```

```
    public static void main (String[] args)
```

```
{
```

```
        P6 obj = new P6();
```

```
}
```

```
}
```

to

```
class P6
```

```
{
```

```
    P6() // default constructor
```

```
{
```

```
}
```

```
    public static void main (String[] args)
```

```
    { P6 obj = new P6();
```

```
}
```

```
}
```

```
}
```

} not visible
for programmer

Note - default constructor always be present without object creation
 after object creation it gets executed.

classmate

Date _____
 Page _____

Accessing Non-static variable with the help of constructor

- 1] We can access Non-static variable in two ways inside constructor
- 1] directly by variable name
 - 2] with the help of this keyword

Class P7

```
{
```

```
int n=30;
```

```
P7(){
```

```
{ S.o.p ln (n);
```

```
n=50;
```

```
S.o.p ln (n);
```

```
int n=100;
```

```
S.o.p ln (n);
```

```
S.o.p ln (this.n);
```

```
}
```

```
public static void main (String[] args)
```

```
{
```

```
P7 obj = new P7();
```

```
//
```

```
S.o.p ln (obj.n);
```

```
}
```

```
test ( )
```

```
{
```

```
P7 obj2 = new P7();
```

```
}
```

Local variable always search first for

Search first for

Local → static → Non-static

→ Initialization of NS var S and look

→ updated value

→ declaration of local variable

→ print local variable

→ if we try to access NS after declaration of local variable we have to use this keyword

We can access non-static from constructor in two ways

- ① directly
- ② this keyword

We can NS from static method in only one way

- ① object reference variable

↓ this is an local variable.

under execution constructor gets executed.

obj is an local variable

→ come for execution

→ search for nonstatic variable.

Heap area ← inside object (inside) non-static

before the execution of constructors
start Non-Static Variable loaded.

Limitations - Local variable

Room anywhere - Static
but for only once

Room anywhere - Non-static
but for more than once

classmate

Date _____
Page _____

```
1st class P8
2nd { // Non-static variable
3rd   int m;

      P8(int n)
      {
        this.m = n;
      }

      public static void main (String [] args)
      {
        P8 obj = new P8(100);
        S.o.p ln (obj.m); // 100

        P8 obj1 = new P8(200);
        S.o.p ln (obj1.m); // 200
      }
    }
```

- Que -
- All static create constructor with all the datatype.
 - Draw diagram of Loading process of non-Static variable.

Note -

- 1] This - points to current object.
- 2] New - create block of memory.

Logging of All Members

Loding

1st SV

2nd SR

3rd SEP

T
↓
B

B

main

Block of memory

1st loading

NS-members
(class) → PS

1] NS-V

2] NS-17

3) NS - 8

pg (int n)

100

this $m=n$

Hand-drawn diagram of a rectangular box. The top-left corner is marked with a dot and a small 'y' above it. The top-right corner is labeled 'P(a|b)'. The bottom-left corner is labeled 'm'. The bottom-right corner is labeled 'phi 100'.

Same process
happens for 2nd
object



↓ 18200

3

```

} PS LinEn)

```

200

This $m = n$

A hand-drawn diagram of a pipe. A horizontal line represents the pipe. Above the line, on the right, is a rectangular box containing the text "P820D". To the left of this box, a downward-pointing arrow indicates a load or connection point. Below the line, on the left, is a rectangular box containing the text "ø 200".

13

$$\underline{1\phi 2}$$

(2)

this points to
current
object

Current

object

- ① First all static members get loaded before the execution of main method begins.

② for loading Non-static member object creation is Mandatory so during the object creation and before execution of Constructor ~~or~~ all Non-static members get loaded similar to static members. ~~the~~ after loading of NS completed Constructor get executed and after the execution of constructor control go to again main method which is under-execution.

* Non-Static Block -

1] A block which is not prefixed by static keyword and declared inside the class block is called as non-static block.

Syntax :

```
{  
    Statements;  
}
```

2] Non static Block does not consist of any name and programmer cannot call non-static Block.

3] Non static Block does not consist of any return type and it does not return any value back to programmer.

4] We can declare more than one Non-Static Block inside the class block.

5] All the non-static block gets executed in top to bottom order.

6] For the execution of Non-Static Block object creation is mandatory.

7] For each object creation all the non-Static Block gets executed only once.

For class - more than one.

- Non-Static Block executed during the object ^{creation} or when main method is under execution.

```
Class PG  
{  
    {  
        S.O.P ln ("NS-Block-1");  
    }  
    {  
        S.O.P ln ("NS-Block-2");  
    }  
    public static void main (String[] args)  
    {  
        PG obj = new PG();  
        PG obj1 = new PG();  
    }  
}
```

- Static Block initializer Block.
- Non-static Block initializer.

```
class Pg
```

```
{
```

static ^{executed before the execution of main method begins.}

① Static Block

```
{
```

```
S.o.p ln ("Static Block");
```

```
}
```

} ^{executed during the object creation}

② Non-static Block

```
{
```

```
S.o.p ln ("Ns-Block");
```

```
}
```

Pg() ^{// executed after the object creation.}

```
{
```

③ Constructor

```
S.o.p ln ("Pg()");
```

```
}
```

```
public static void main (String[] args)
```

```
{
```

```
Pg obj = new Pg();
```

```
}
```

```
}
```

* Note -

For the execution of non-static Block and Constructor object creation is mandatory.

* Non-static advantage - We can load Non-Static member for more than once. (but once for each object creation).

Que -

- 1] Can we access a non-static variable from a non-static Block.
- 2] Can we initialize non-static variable from non-static Block.
in How many ways.

* Non - static Method -

- 1] A method declared inside the class block and not prefixed with by static keyword is called Non-static method.
- 2] We can access a non static method with the help of Object reference variable.
- 3] For loading Non static method object creation is mandatory.
- 4] We can declare more than NS method inside Class Block.

Assignment No.

Q.1 Can we access a non-static variable from a non-static Block

```

→ class P1
{
    int n;
    public static void (String[] args)
    {
        P1 class = new class P1();

        {
            n = 10;
            this.n = 30;
            s.o.p.println(this.n);
        }
    }
}
  
```

Q.2 Can we initialize non-static variable from non-static block in How many ways

```
→ class P1
{
    int n = 100;
    {
        int m = 10;
        s.o.p(n);
        this.n = 200;
        s.o.p(n (this.n));
    }
}
```

```
P1()
{
    s.o.p(n);
    n = 300;
    s.o.p(n (this.n));
}
```

```
public static void main (String[] args)
{
    P1 obj = new P1();
}
```

Q.3 Can we access Non Static variable from non-static method (in How many way)

```
→ class P1
{
    int a = 10;
    {
        test('b');
    }
}
```

calling main method
is imp

without calling main method
won't get executed.


```

public void test (char a)
{
    this.a = a;
    S.O.P.ln (this.a);
}

```

```

public static void main (String [] args)
{
    P1 obj = new P1();
}

```

Q.4 Can we initialize non static variable from non static method
(In How many way)

→ Class P1

```

{
    Long
    int a = 67891;

    P1 (char a)
    {
        S.O.P.ln (a);
        S.O.P.ln (this.a);
        this.a = a;
        S.O.P.ln (this.a);    this.test (300);
    }

    public static void main (String [] args)
    {
        P1 obj = new P1(0);
        obj.test (200);
        test (100);
    }

    public void test (double b);
    {
        a = b; a = (long) b;
    }
    S.O.P.ln (a);    S.O.P.ln (this.a);
}

```

this reference is not used in static members.

classmate

Date

Page

Q.5 Can We access Non-static method from non-static Block (in How many ways)

→

```
class P1
{
    int n;
    public static void test (int a)
    {
        this.n = a;
        S.o.P.ln (this.n);
    }
    {
        int a = 100;
        a = test (10);    S.o.P.ln (a);
    }
    public static void main (String[] Args)
    {
        P1 Obj = new P1();
    }
}
```

- yes
- Directly also
- By using this also

Q.6 Can We access Non-static method from no argument constructor (in How many ways)

→

```
class P1
{
    int a = 2;
    public void test (int a)
    {
        this.a = a;
        S.o.P.ln (a);
    }
}
P1()
{
    test (100);
    this.test 100;    S.o.P.ln (a);
}
}
```



```

public static void main (String [] args)
{
    P1 obj = new P1();
}

```

Q7 Can we access non-static method from parameterized constructor (in How many way)

→ Class P1

```

{
    char a; char data
    return
    public static void test (char a)
    {
        this.a = a;
        S.o.pln (this.a);
    }
    Return this.a;
}

P (int a)
{
    a = test (20);
    S.o.pln (a)
    S.o.pln (test 10);
}

```

```

public static void main (String [] args)
{
    P1 obj = new P1 (101);
}

```

5 where static get stored

classmate

this keyword first search for
① Non static if it is not present
then ② Static

* Loading process of Non-Static Members -

1] In order to load non-static members of class object creation is Mandatory.

Class P11

{

int a = 123;

loaded - 1

P11()

{ S.o.Pln("P()");

}

start
- execution of
constructor
after the execution
of constructor
control transfer
to main method

{

S.o.Pln("NS-Block");

}

get executed - 3

public void test()

{

S.o.Pln("test()");

}

loaded - 2

public static void main (String [] args)

{

P11 obj = new P11();

S.o.Pln(obj.a);

obj.test();

}

Loading of class depends on static member - does not belong to non-static
and non-static loading depends on object creation.

* Loading of object $\rightarrow$ Non-static members
Loading of class $\rightarrow$ Static members
Loading of NS $\xrightarrow{\text{depends on}}$ object creation

classmate

Date _____

Page _____

- 2] During the loading process of Non-static Members Non-static variable gets loaded first once after the loading process of non-static variable is completed non-static method gets loaded next
- 3] once after the loading of Non-static v and Non-static Method is completed all the non-static blocks get executed in top to bottom order.
- 4] Once after the execution of Non-static block is completed then constructor of the class get executed.

* What are the advantages of constructor.

- 1] With the help of constructor we can create multiple objects for a class.
- 2] With the help of constructor we can initialize a non-static members after object creation.
- 3] With the help of constructor we can access both static and Non-static members.

* OOPS (Object oriented programming)

1] Object oriented programming is a concept or a structure where we take all the real time examples and convert into a programmatical world.

2] JAVA is "object oriented programming language".

3] The real time object consists of two (2) elements.

1] properties.

2] behaviours.

4] properties are converted into variables.

5] Behaviours are converted into methods.

6] Variables and methods are the members of the class.

7] Hence class is called as Blueprint of the object.

(Real time examples)

8] Object oriented programming consists of following pillars.

1] Encapsulation

2] Inheritance

3] Polymorphism

4] Abstraction

1] Encapsulation -

It is a process of hiding. Achieving the data hiding is called as Encapsulation.

We can perform data hiding with the help of Private keyword.

* Data hiding -

Hiding the data (value or property) from outside world is called as data hiding.

• private -

- 1] It is an keyword in Java.
- 2] IF we prefixed any variable with private keyword we cannot access the private variable from another / different class.
(modification (changing the value)
2] printing the value is not possible)

In order to access the private data (private variable) we need to create two methods

- 1] Getter method
- 2] Setter method

1] Getter method -

1] It is used to access the value present inside the private variable

2] The return type of Getter method is always depends on the variable which is made as private.

Method header of Getter method -

depends on variable which is private

```
modifier [return type] getPrivate_variable_name()  
{  
    return Private_variable_name;  
}
```

Getter method

Not private → SOP In C/C++

We can use getter and setter for accessing private variable.

classmate

Date _____
Page _____

2] Setter method.

1] It is used to modify the value which is present inside the private variable.

2] Setter method should be parameterized method.

method header of setter method :-

```
Modifier return type set_private_variable_name (datatype var_name)
{
    this.private_variable_name = var_name;
}
```

Passing value

Class Customer

{

String A_name;

String A_number;

private double A_balance;

// getter method for A_balance

public double get_balance()

{

return this.A_balance;

}

// setter method for A_balance

public void set_A_balance (double balance)

{

this.A_balance = balance;

}

Customer (String name, String no, double balance)

{

this.A_name = name;

this.A_number = no;

this.A_balance = balance;

}

is present
method
(destructor)

```

public void A_details
{
    S.O.P ln (this.A_name);
    - " — (this.A_number);
    - " — (this.A_balance);
}

```

```

public void loan()
{
    S.O.P ln (this.A_name + " has applied for loan");
}
}

```

```

class Bank
{
    static String B_name = "SBI";

```

```

    public static void main (String[] args)
    {

```

```

        Customer c1 = new Customer ("Jay", "12345678", 1000.0);

```

```

        // S.O.P ln (c1.A_balance); // CTE becz variable is private
        S.O.P ln (c1.get_balance()); // getter method (no argument)

```

```

        // c1.A_balance = 12344.122; // CTE private variable

```

```

        c1.A_balance (12344.123); // setter method
                                   (parameterized)

```

```

        S.O.P ln (c1.get_balance());

```

```

    }

```

```

    // A_balance is still private - we can access by setter and
    // Getter method only.
}

```

return

$n * \text{fact}(n-1)$

JAVA

classmate

Date _____
Page _____

* Introduction -

Java was developed by a team of computer professionals under the guidance of James Gosling of sun microsystem in 1991. They wanted to give a suitable name. Initially they called it "Oak" after seeing an oak tree from their window, but after some weeks they were discussing another name and were sipping Java coffee, so one of them suggested the name "JAVA".

Generally a language is either compiled or interpreted but java is both compiled as well as interpreted. First a java program is compiled and comes in the form of "JAVA byte code" (which is an intermediate code). Then this java byte code is run on interpreter to execute a program.

* Features of Java -

① Platform Independent - You can develop the java program on any computer system and the execution of the program is possible on any other computer system loaded with JVM. For example, you can write and compile the java program on windows 98 and execute the compiled program on JVM of the macintosh operating system.

② Object oriented - Java is a true object-oriented language, which provides a platform to develop an effective and efficient application and program real life complexities. Most of the object-oriented concepts in java are inherited from C++ which makes it easy for traditional programmers to understand it.

Easy to Learn -

because it is (syntax) similar to C and C++ and most of the complex parts of C/C++ have been excluded including operator overloading, multiple inheritance and pointers. The important addition in Java is automatic memory allocation and de-allocation.

* Robust - Java programmers ^{not need to} need not remember to de-allocate memory in programs since there is a ^{garbage} ~~garbage~~ collection mechanism which handles de-allocation of memory. It provides a robust exception handling mechanism to deal with both expected and unexpected errors at run time.

• JVM (Java Virtual machine):

The JVM plays the main role to making Java portable. It provides a layer of abstraction between the compiled Java program and the hardware platform and operating system. The JVM is central to Java's portability because compiled Java programmes run on the JVM, independent of whatever hardware is used.

Program → Compiled → Bytecode → JVM → Interpreter
↓
machine code

- Java is case sensitive.
- White space is meaningless except inside string literals.

classmate
Date _____
Page _____

* Separators in Java -

- 1] **parentheses ()** - Generally it encloses arguments in method definitions or calling and used to delimit test expressions control statements.
- 2] **Braces {}** - Defines blocks of code and it also defines automatically initializers arrays.
- 3] **Square Bracket []** - declare array types.
- 4] **Semicolons ;** - Semicolons are used to terminate statements.
- 5] **single comma ,** - It is separates successive identifiers in variables declarations.
- 6] **single dot .** - It selects a method from an object and separates package names from sub-package and class names.
- 7] **Colon :** - It is used in loop labels.

* 2D Array

* 2D Array -

```
type[][] arrayName = new type[rows][columns];
```

```
int[][] numbers = new int[3][5];
```

		columns					
		0	1	2	3	4	5
row ↓	0						
	1				4		
	2						

Example:

```
Scanner sc = new Scanner(System.in);
```

```
int rows = sc.nextInt();
```

```
int cols = sc.nextInt();
```

```
int[][] numbers = new int[rows][cols];
```

→

```
// Input
```

```
for (int i = 0; i < rows; i++) {
```

```
    // columns
```

```
    for (int j = 0; j < cols; j++) {
```

```
        numbers[i][j] = sc.nextInt();
```

```
    }
```

```
}
```

→

```
// output
```

```
for (int i = 0; i < rows; i++) {
```

```
    for (int j = 0; j < cols; j++) {
```

```
        s.o.p.println(numbers[i][j] + " ");
```

```
    }
```

```
s.o.p.println();
```

```
}
```

Q3. Take a matrix as input from the user. Search for a given number x and print the indices at which it occurs.

* Bit Manipulation -

operations.

left shift

$n \ll i$

$2 \ll 1$

$010 \ll 1$ - position of shifting

↓
100 Final output

Binary नं मध्ये

left shift or
right shift कर
शकतो

right shift

$n \gg i$

$2 \gg 1$

$010 \gg 1$

001

- ① Get Bit ② Set Bit ③ Clear Bit ④ Update Bit

1] Get Bit -

get the 3rd bit (position = 2) of a number n . ($n = 0101$)

BitMask : $1 \ll i$

Manipulation in
number.

operation : AND

0 1 0 1
3 2 1 0
position

Horizontal and vertical regions + rows
and columns

* table - A table is an organized collection of

Fields - columns in a table.

records - rows in a table.

* Index - It is data structure that provides quick lookup of data in a table column.

It enhances the speed of operations accessing data from a database table.

Unique and Non-Unique Index -

Clustered and Non-clustered index.

* Data Integrity?

→ It is the assurance of accuracy and consistency.

* What is View?

A View in SQL is a virtual table based on the result set of an SQL statement.

It contains rows and columns just like real tables.

```
int n = arr.length;
```

```
int total = 1;
```